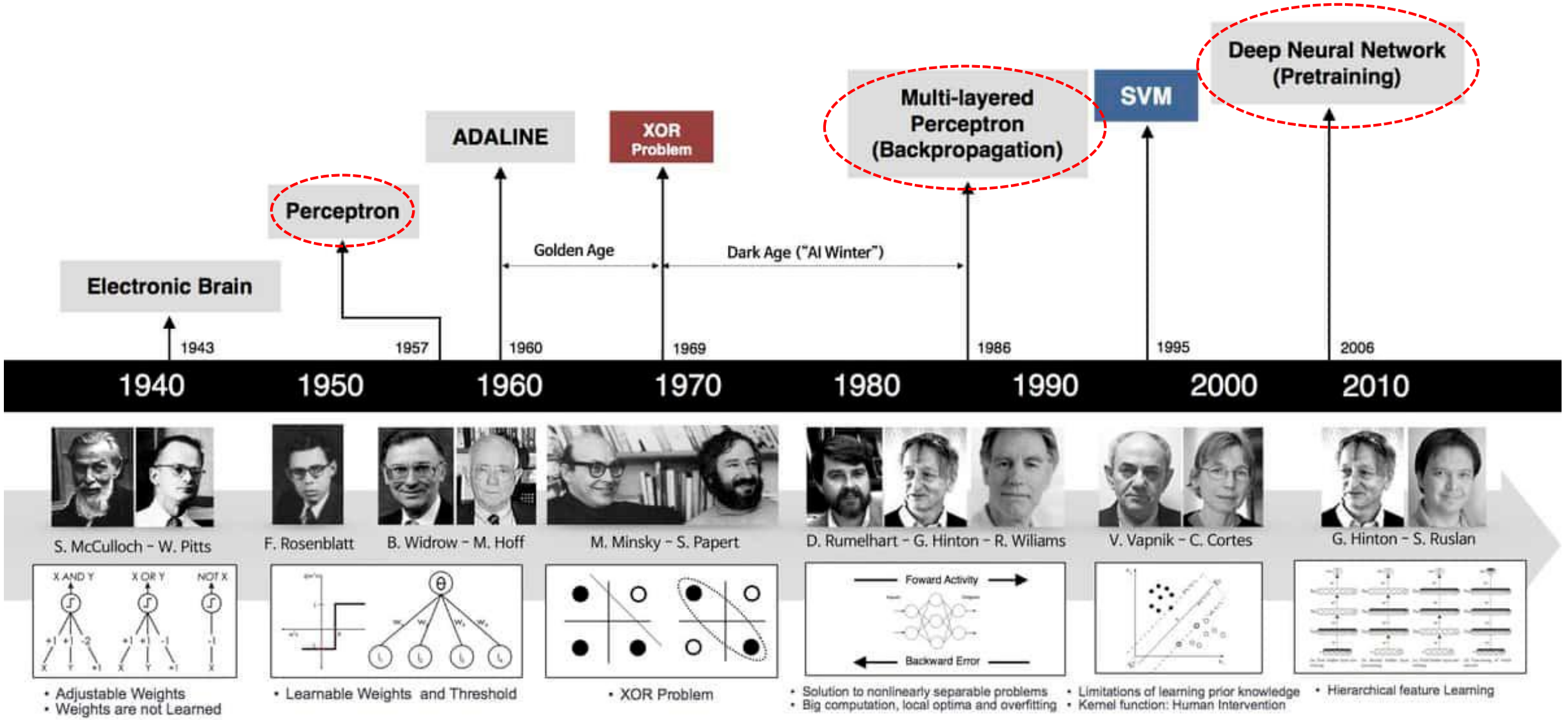
2. Basic Architecture

DNN

Evolution of deep learning

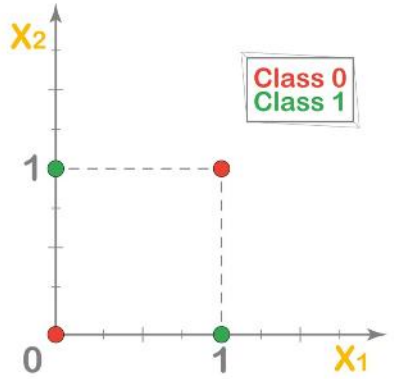
https://fiveable.me/deep-learning-systems/unit-1/historical-context-evolution-deep-learning/study-guide/ALVCX29Pf2dG574E

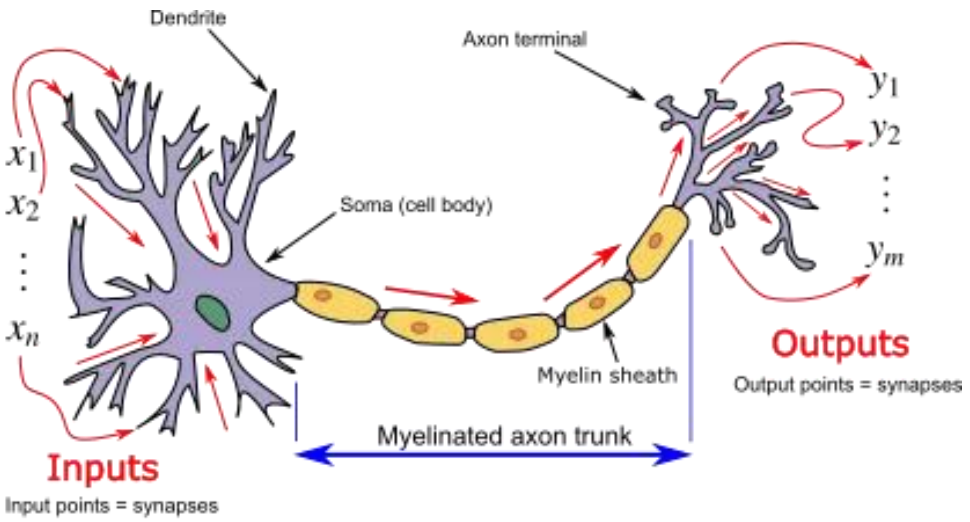


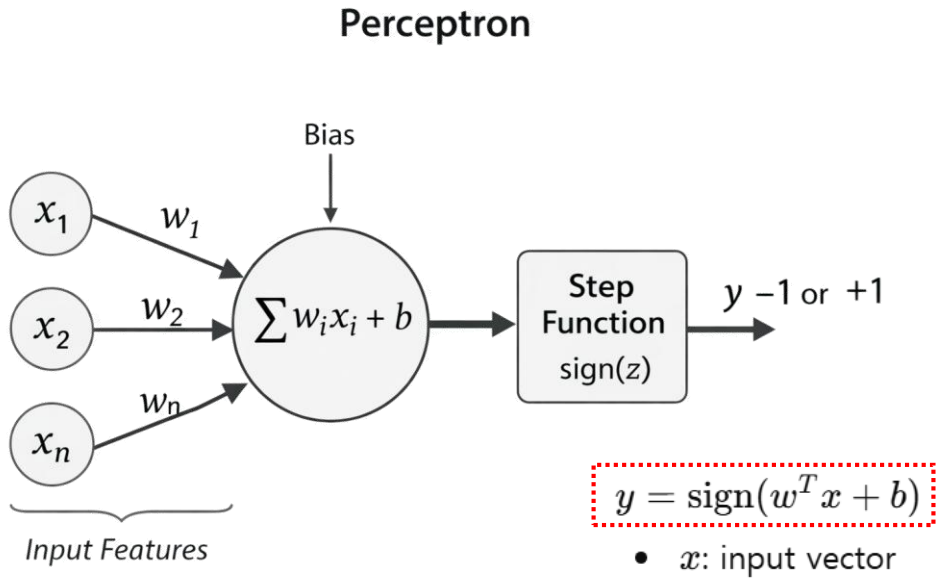
Perceptron

https://www.analyticsvidhya.com/blog/2022/10/multi-layer-perceptrons-notations-and-trainable-parameters/
https://anyantudre.medium.com/from-biological-to-artificial-neurons-theory-behind-perceptrons-d0ccc5bcf1eb

- The **simplest form of an artificial neuron**
- **Binary classification** using a weighted sum and a threshold
  - Can solve linearly separable problems (e.g., AND, OR)
  - Can not solve non-linear problems (e.g., XOR)





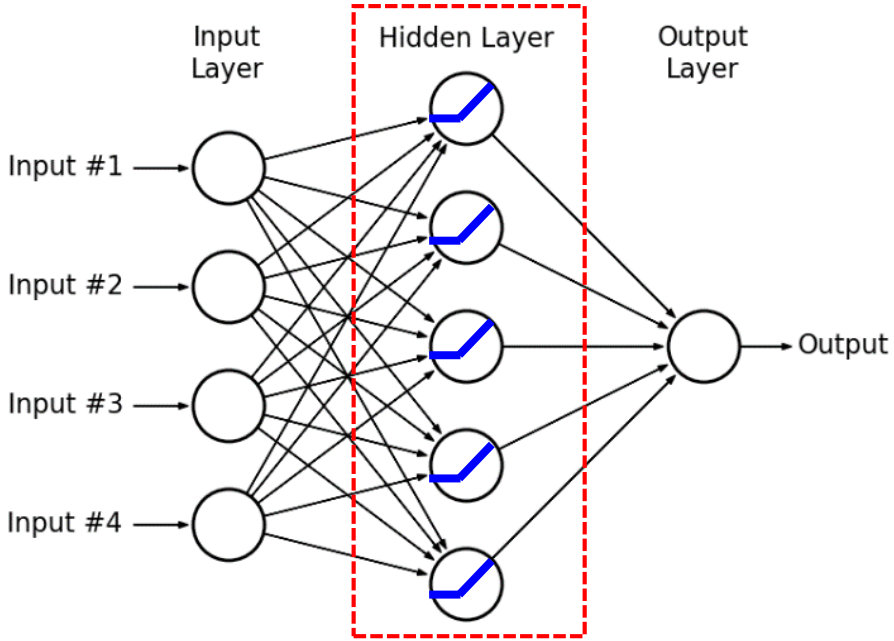


$$y = \text{sign}(w^T x + b)$$

- x: input vector
- w: weight vector
- b: bias
- y: output label (-1 or +1)

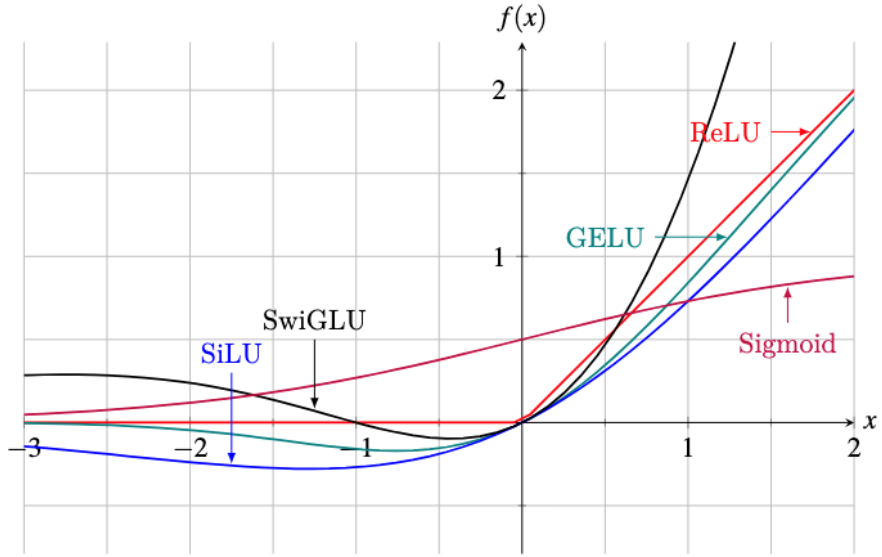
MLP (Multi-Layer Perceptron)

- **Feedforward neural network** with one or more **hidden layers**.
- **Learning non-linear relationships** through multiple transformations.
  - Can solve non-linear problems like XOR
- **Key components**
  - Non-linear activation functions in hidden layers
  - Trained using backpropagation



https://sonsnotation.blogspot.com/2020/11/5-multilayer-perceptrons-mlps.html

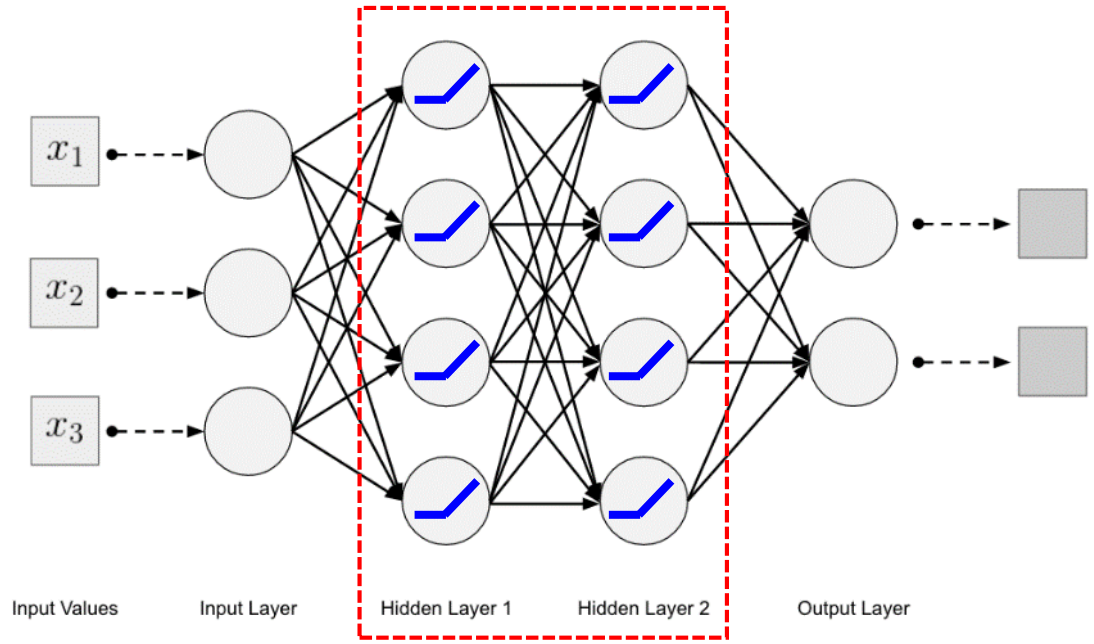
Activation functions



https://machinelearningmastery.com/linear-layers-and-activation-functions-in-transformer-models/

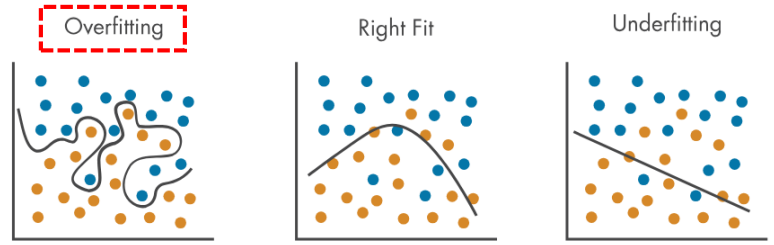
DNN (Deep Neural Network)

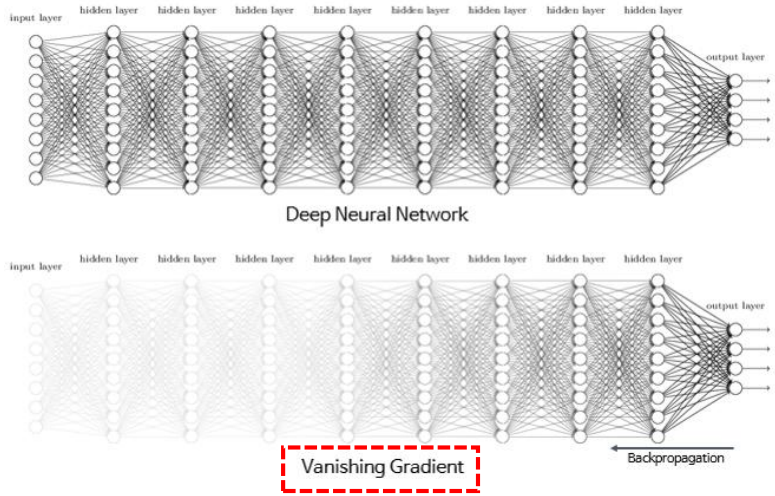
- Essentially an **MLP with many hidden layers**.
  - Deep MLP + scalable architecture
- **Common Architectures**
  - Fully Connected Networks (FCNs) – Classic DNNs for classification tasks
  - Convolutional Neural Networks (CNNs) – DNNs specialized for image data
  - Recurrent Neural Networks (RNNs) – DNNs for sequential/time-series data
  - Transformers – Deep attention-based networks for NLP and Vision



https://toolmyai.com/deep-learning-and-neural-network-trends-in-2024/

- **Challenges**
  - Overfitting
  - Vanishing gradients
    (solved by ReLU, BatchNorm, Residuals)



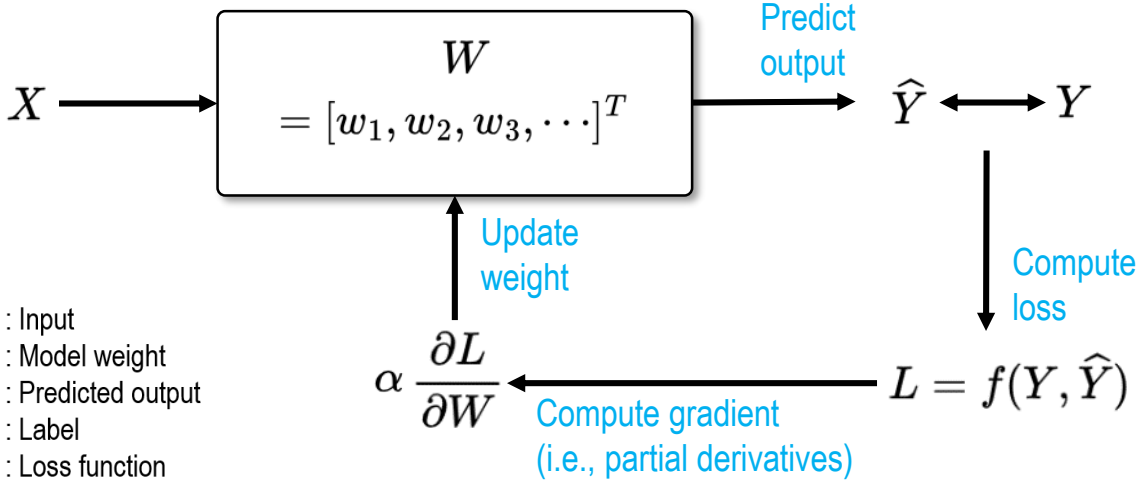


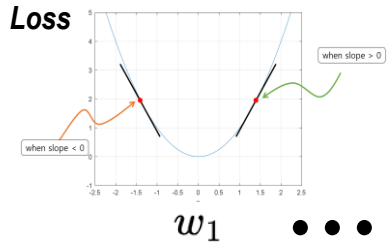
https://www.mathworks.com/discovery/overfitting.html
https://minkithub.github.io/2020/05/27/everydeeplearning7/

Summary

	Perceptron	MLP (Multi-Layer Perceptron)	DNN (Deep Neural Network)
Definition	Single-layer neural unit	Shallow neural network with one or two hidden layers	Deep network with many hidden layers
Layer Depth	1 (Input → Output)	2–3 (Input → Hidden → Output)	3+ layers (multiple hidden layers)
Activation Function	Step function	Sigmoid, ReLU, Tanh	ReLU, GELU, Softmax, etc.
Training Method	Simple rule-based update	Backpropagation	Backpropagation + Optimization techniques
Representation Power	Low	Moderate	High (Hierarchical feature learning)
Use Cases	Linearly separable tasks	Small classification tasks	Image, speech, NLP, medical data, etc.
Computation	Very low	Low to moderate	High (GPU/TPU often required)
Interpretability	High	Medium	Low
Scalability	Very limited	Moderate	Highly scalable (large-scale models)
Regularization	Not needed	Often used (dropout, L2)	Essential (dropout, batch norm, etc.)
Historical Relevance	Origin of neural networks (1958)	Rebirth of neural nets in 1980s	Backbone of modern deep learning

Back-propagation



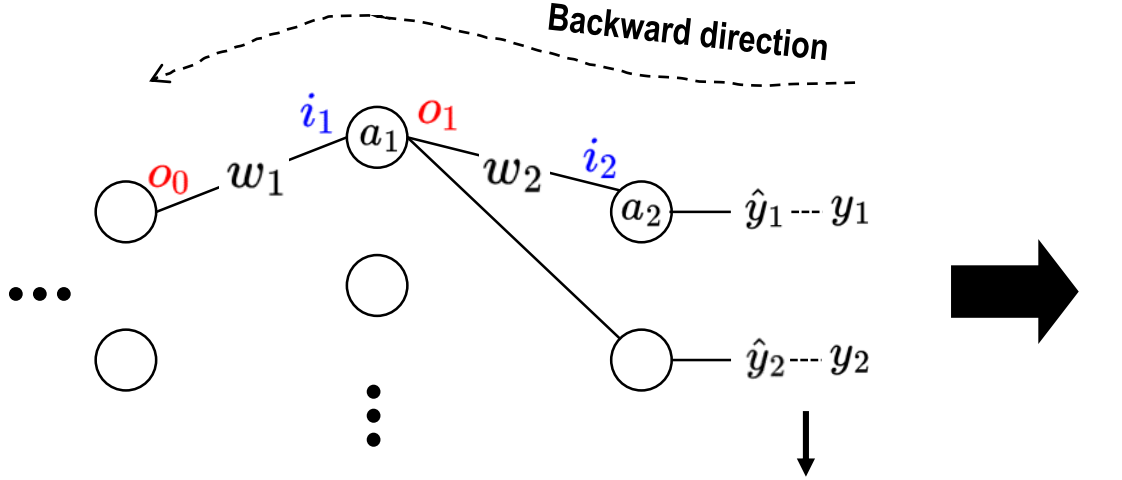


Update weight

Compute gradient

$$W_{t+1} = W_t - \alpha \frac{\partial L}{\partial W_t}$$

$$\begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ \vdots \\ w_n \end{bmatrix}_{t+1} = \begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ \vdots \\ w_n \end{bmatrix}_{t} - \alpha \begin{bmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \frac{\partial L}{\partial w_3} \\ \vdots \\ \frac{\partial L}{\partial w_n} \end{bmatrix}_{t}$$



$$L = (y_1 - \hat{y}_1)^2 + (y_2 - \hat{y}_2)^2 + \cdots$$

e.g., MSE (Mean Squared Error)

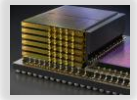
$w_2 \rightarrow i_2 \rightarrow \hat{y}_1 \rightarrow L$

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial \hat{y}_1} \frac{\partial \hat{y}_1}{\partial i_2} \frac{\partial i_2}{\partial w_2} = 2(\hat{y}_1 - y_1) \cdot a'(i_2) \cdot o_1$$

$w_1 \rightarrow i_1 \rightarrow o_1 \rightarrow i_2 \rightarrow \hat{y}_1 \rightarrow L$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}_1} \frac{\partial \hat{y}_1}{\partial i_2} \frac{\partial i_2}{\partial o_1} \frac{\partial o_1}{\partial i_1} \frac{\partial i_1}{\partial w_1} + \cdots = 2(\hat{y}_1 - y_1) \cdot a'(i_2) \cdot w_2 \cdot a'(i_1) \cdot o_0 + \cdots$$

On memory

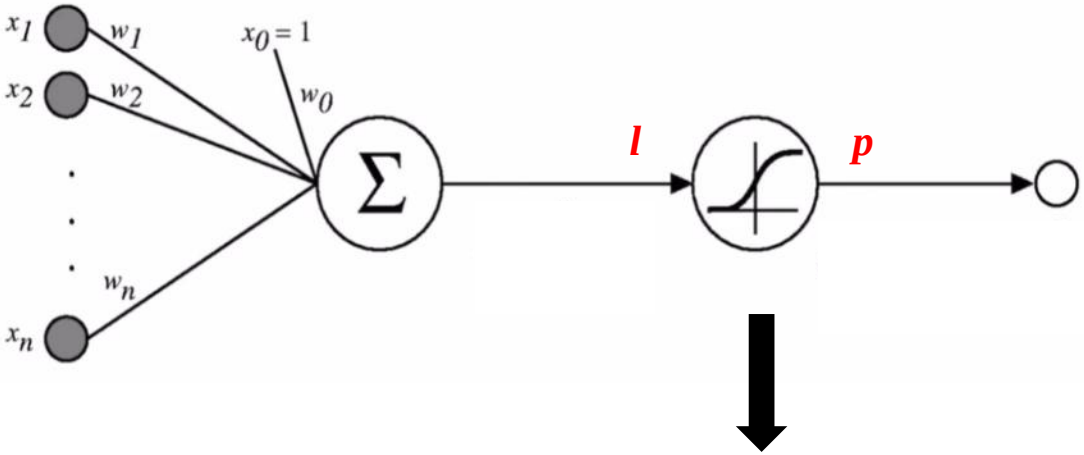


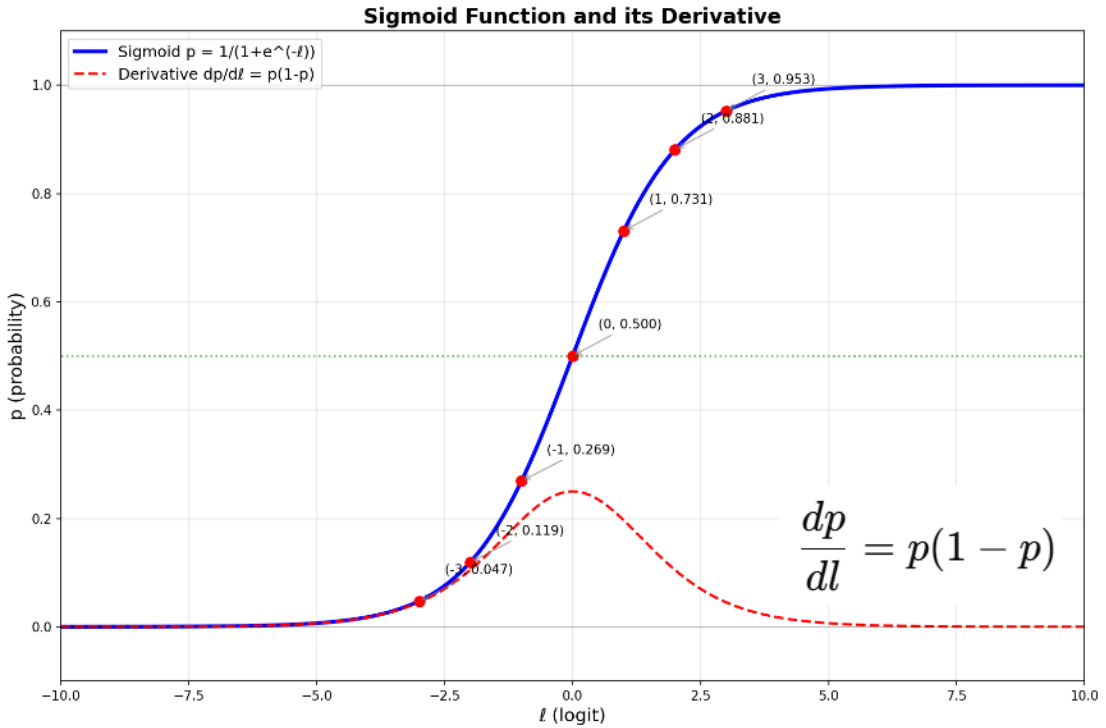
- Weights
- Derivative of the activation function
- Activation values (after feed-forward)
- Gradient history

When does the loss stop changing?

Sigmoid (Logistic regression / Binary classification)

- Often interpreted as a **probability-like** output
  - Any real-valued number into the range (0, 1)
- **Differentiable** at all points
  - Important for backpropagation





- **Logit** (i.e., l) = ln(odds), where odds = $\frac{p}{1-p}$
  - p : The **probability** of an event occurring, a value between 0 and 1.

$$l = ln\left(\frac{p}{1-p}\right)$$

$$e^{l} = \frac{p}{1-p}$$

$$e^{-l} = \frac{1}{p} - 1$$

$$p = \frac{1}{1+e^{-l}}$$

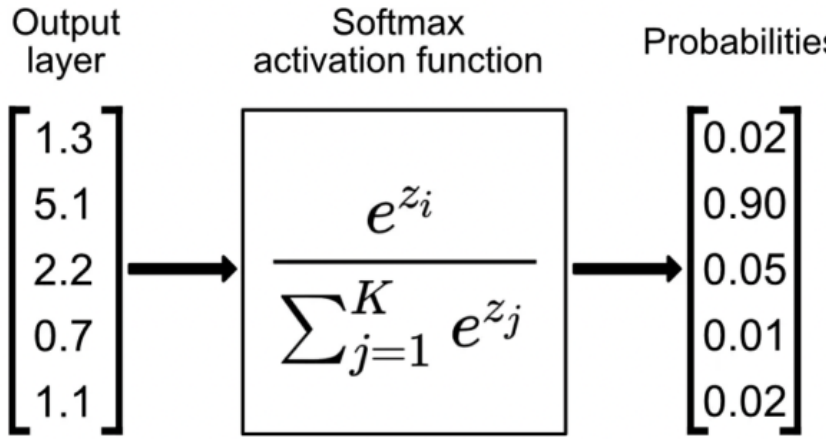
Sigmoid function

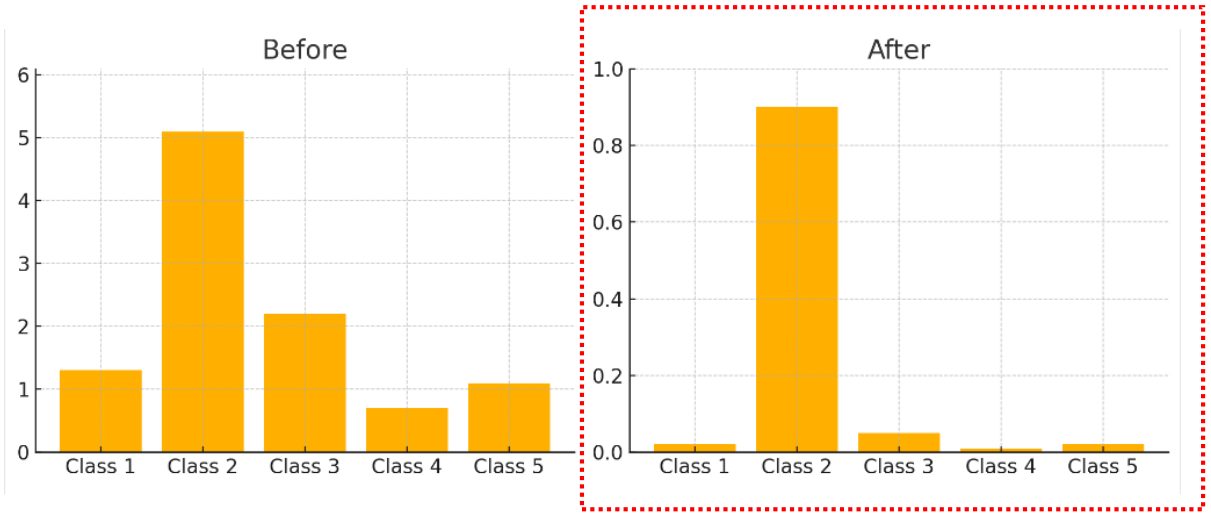
Softmax (Multi-class classification)

https://www.singlestore.com/blog/a-guide-to-softmax-activation-function/

- Mathematical function that **converts raw scores (i.e., logits) into probabilities**
  - The exponential function amplifies differences between scores
  - A higher score gets a higher probability
  - A lower score gets suppressed

- Used in the **'output layer'** of the **multi-class classification**





$$\frac{e^{z_i - m}}{\sum_{j=1}^{K} e^{z_j - m}}, \quad m = \max z_j$$

Loss

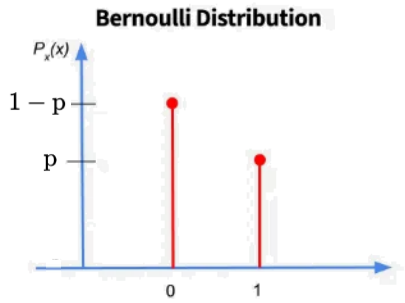
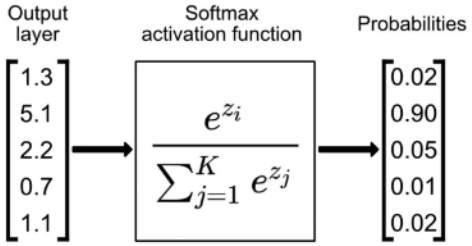
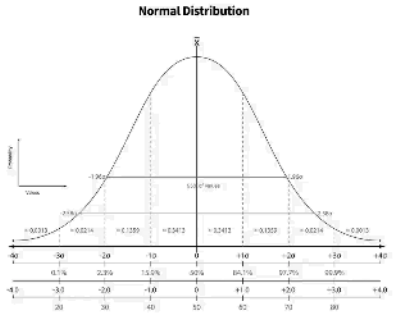
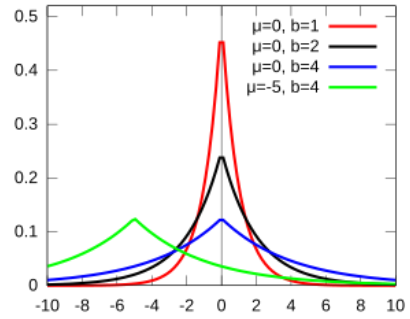
- **Toy examples of the loss computation**

	Binary Cross Entropy (BCE)	Cross Entropy (CE)
Task	Binary Classification (Cat vs Not Cat)	4-class classification (Cat, Dog, Car, Bird)
Ground truth	$y = 1$	$y = [1, 0, 0, 0]$ ← **One-hot**
Prediction	$\hat{y} = 0.8$	$\hat{y} = [0.7, 0.2, 0.05, 0.05]$
Loss	$\begin{aligned} \text{BCE} &= -\left(y \log \hat{y} + (1-y) \log(1-\hat{y})\right) \\ &= -\left(1 \cdot \log 0.8 + 0 \cdot \log 0.2\right) \\ &= -\log 0.8 \end{aligned}$	$\begin{aligned} \text{Cross Entropy} &= -\sum_{i=1}^{C} y_i \log \hat{y}_i \\ &= -(1 \cdot \log 0.7 + 0 + 0 + 0) \\ &= -\log 0.7 \end{aligned}$

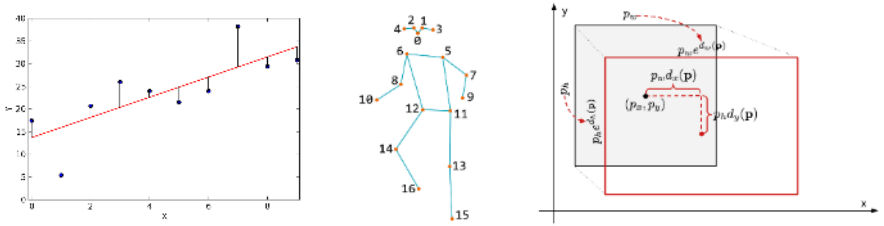
Network output distribution and corresponding loss

$$p(W|\hat{y}) = \frac{p(\hat{y}|W)\,p(W)}{p(\hat{y})}$$

Likelihood

	Bernoulli	Categorical	Normal	Laplace
Observed data ($\hat{y}$) from a model parameter (W)				
Likelihood **i.e., MLE (→ *Maximize*)**	$f(\hat{y}\|W) = \hat{y}^{y}(1-\hat{y})^{1-y}$	$f(\hat{y}\|W) = \prod_{k=1}^{K} \hat{y}_k^{y_k}$	$f(\hat{y}\|W) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(\hat{y}-\mu)^2}{2\sigma^2}}$	$f(\hat{y}\|W) = \frac{1}{2b} e^{-\frac{\lvert\hat{y}-\mu\rvert}{b}}$
NLL **(Negative Log Likelihood)**	$-\log f(\hat{y}\|W) = -y\log\hat{y} - (1-y)\log(1-\hat{y})$	$-\log\ f(\hat{y}\|W) = -\sum_{k=1}^{K} y_k \log\hat{y}_k$	$-\log f(\hat{y}\|W) = (\hat{y}-y)^2$	$-\log f(\hat{y}\|W) = \lvert\hat{y}-y\rvert$
i.e., Loss (→ *Minimize*)	BCE (Binary Cross Entropy)	CE (Cross Entropy)	MSE (Mean Squared Error)	MAE (Mean Absolute Error)
Applications	Binary classification	Multiclass classification	Regression	

Regression vs. Classification

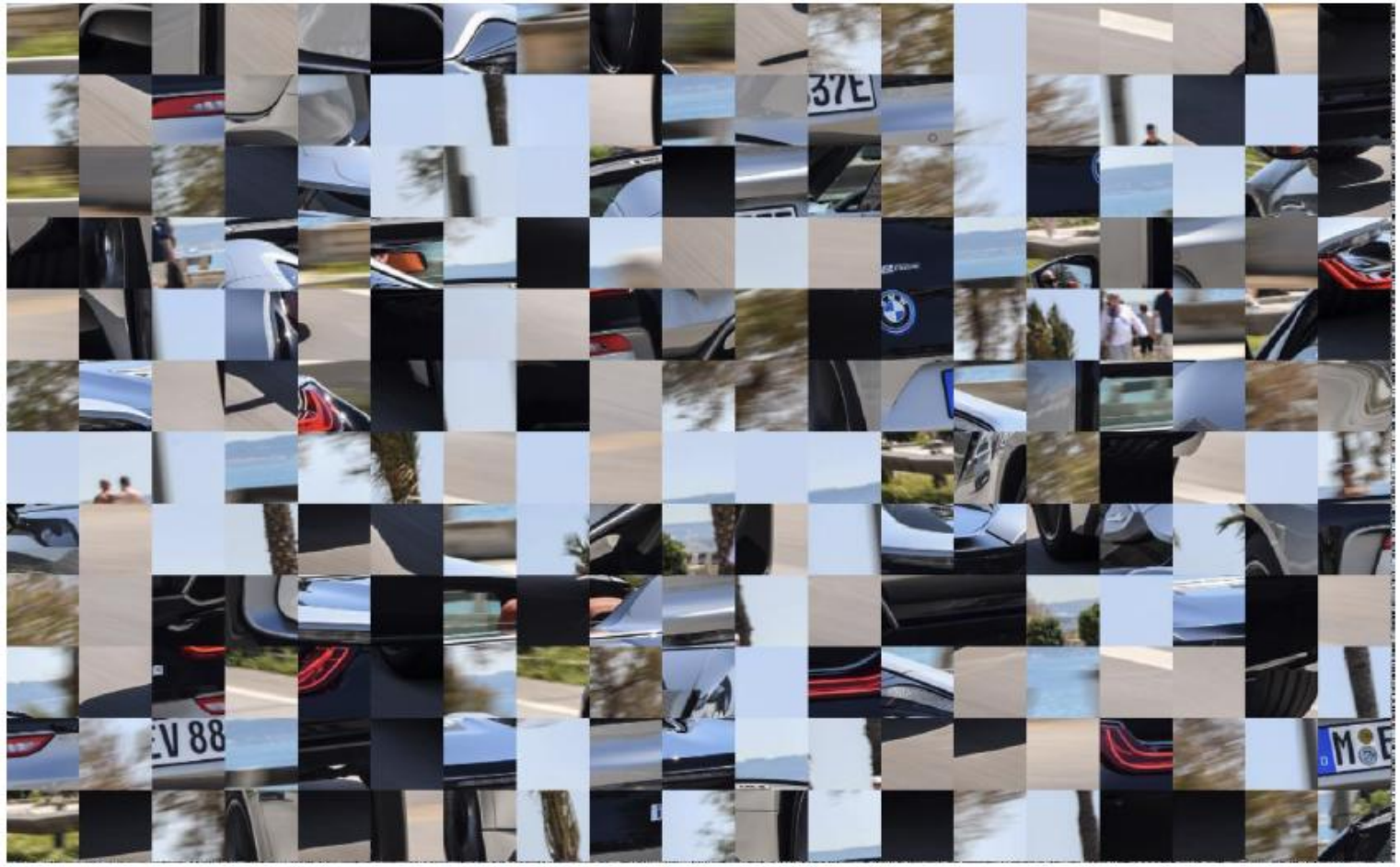
	Regression	Classification
Output	Continuous values	Discrete class labels
Activation	Linear in general	Sigmoid / Softmax
Loss	MSE, MAE	Cross entropy
Metrics	RMSE	Accuracy, F1-score, mAP
Representation	Real numbers	Probability distribution
Applications	Estimation, Pose, Bounding box 	Object category, Segmentation 

https://philosopher-chan.tistory.com/461
https://www.design.kyushu-u.ac.jp/~eigo/behavior_senpai.html
https://better-tomorrow.tistory.com/entry/Bounding-box-regression
https://viso.ai/deep-learning/object-detection/
https://viso.ai/deep-learning/panoptic-segmentation/

CNN

Statistical pattern matching of DNN

- **Deep learning learns statistical correlation, not visual semantics**



Convolution (1/3)

- **Convolution** vs. Cross-correlation vs. Autocorrelation

Mirroring

↓

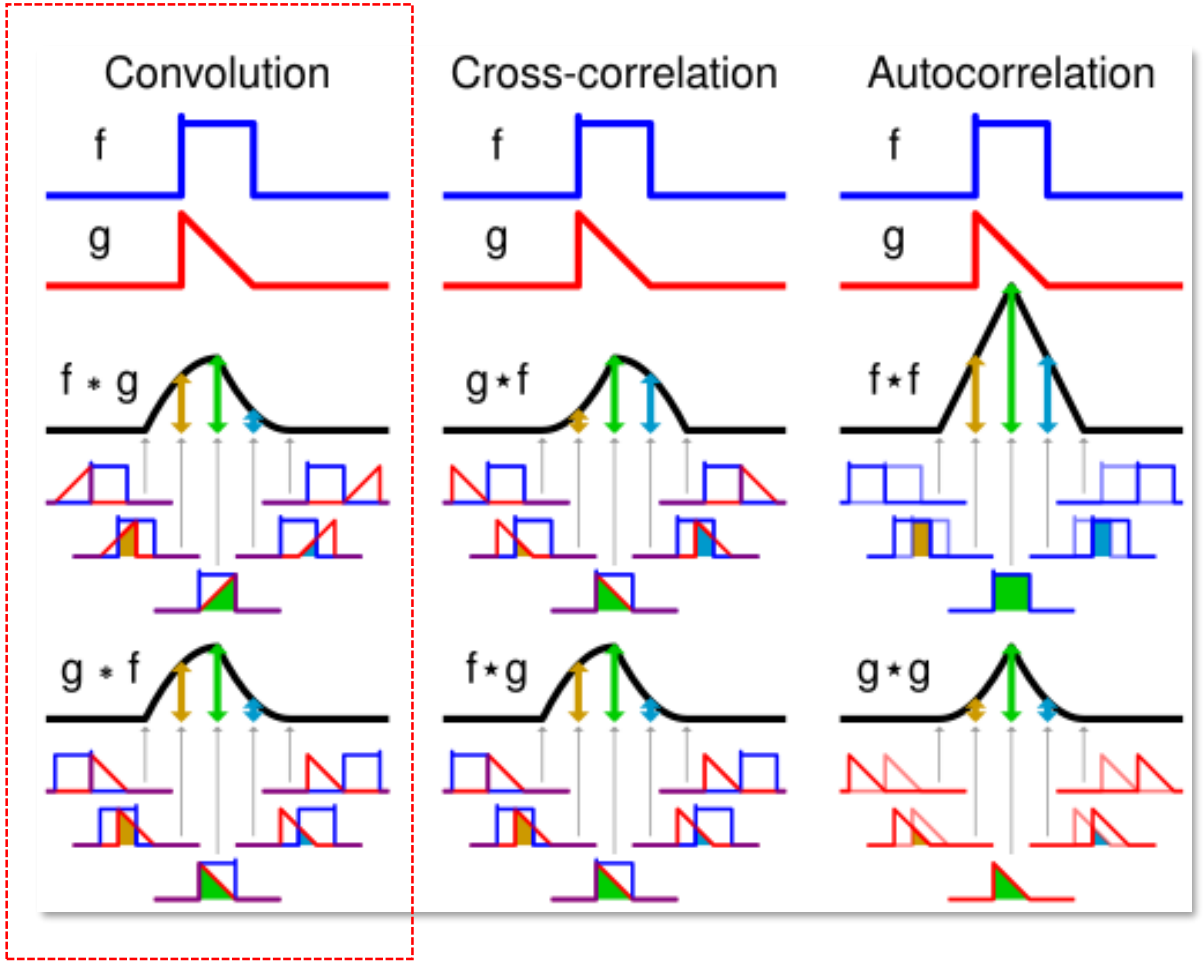
Moving

↓

Multiplication

↓

Addition



https://en.wikipedia.org/wiki/Convolution

Convolution (2/3)

https://janghan-kor.tistory.com/673
https://rbe549.github.io/spring2025/hw/hw0/

- **Conventional handmade image processing filters**

Edge detection filters

Sobel

1	0	-1
2	0	-2
1	0	-1

1	2	1
0	0	0
-1	-2	-1

Scharr

3	0	-3
10	0	-10
3	0	-3

3	10	3
0	0	0
-3	-10	-3

Prewitt

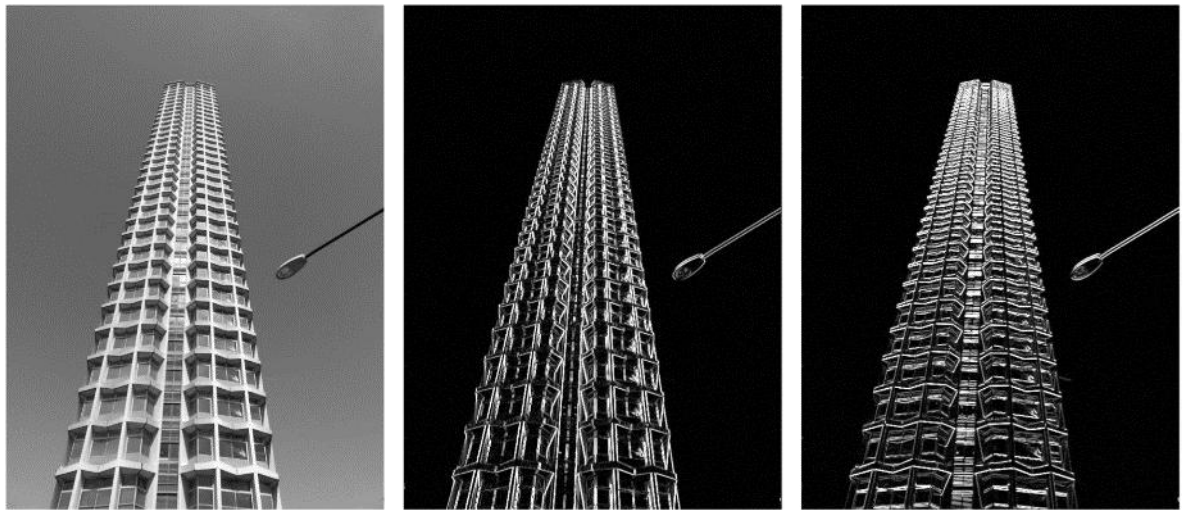
1	0	-1
1	0	-1
1	0	-1

1	1	1
0	0	0
-1	-1	-1

Roberts

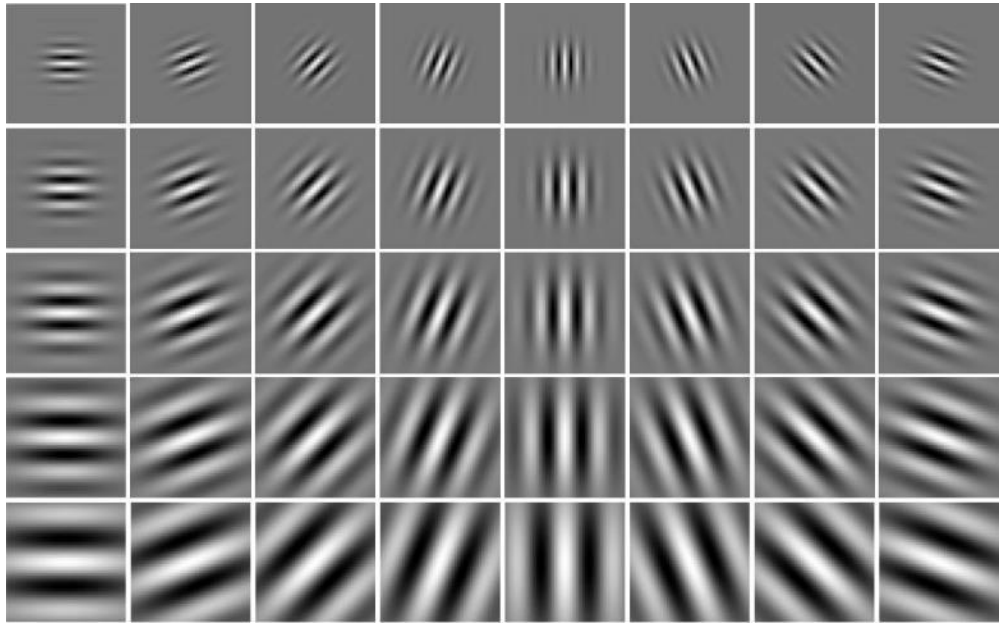
0	1
-1	0

1	0
0	-1

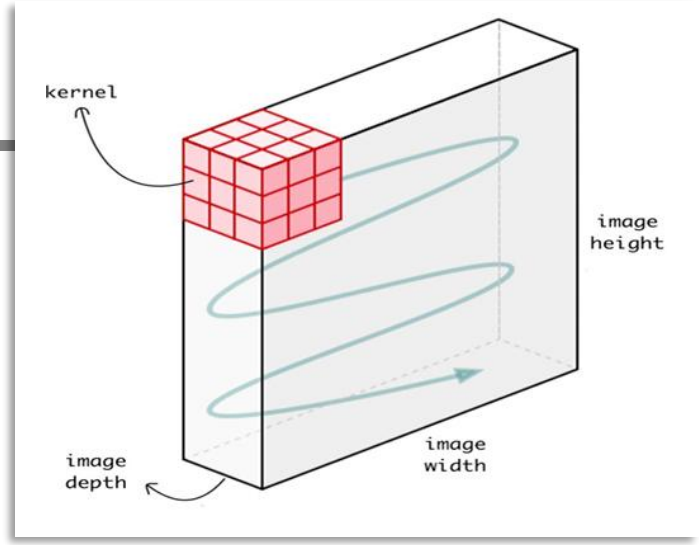


original | vertical Sobel filter | horizontal Sobel filter

Directional filters

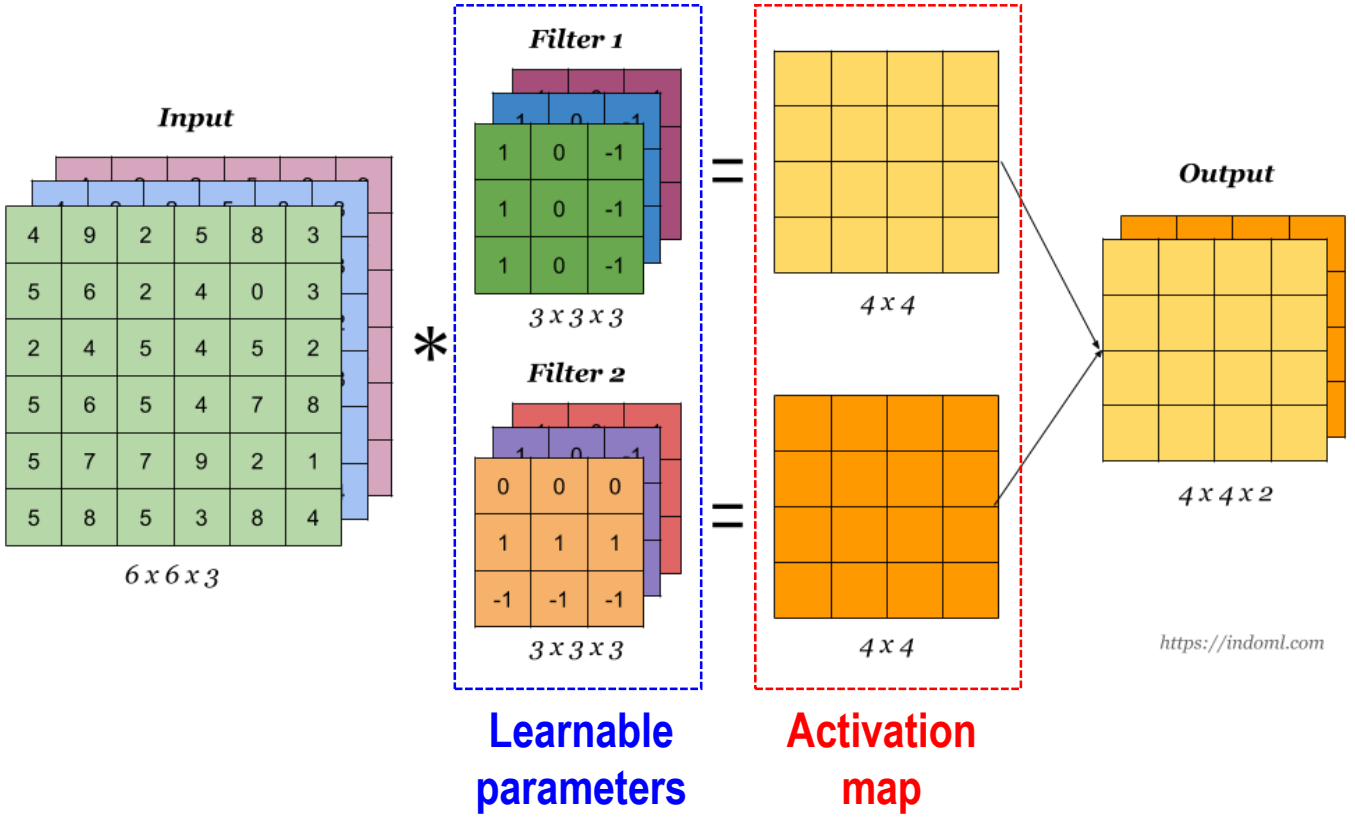


Convolution (3/3)



- **Convolution in CNNs**
  - A process where a small filter (or kernel) slides over an image
  - Thereby, it extracts important feature like edges, textures or patterns
  - It looks at local regions instead of the whole image at once

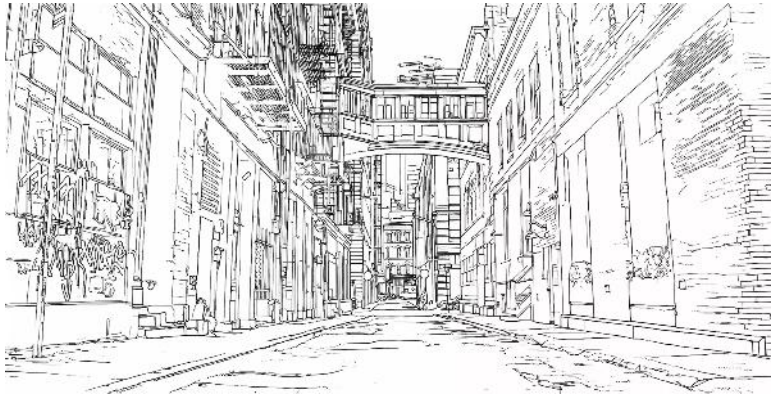
Multi-channel convolution



Why convolution ? (From Fully Connected Network to CNNs)

- **Parameter sharing**
  - The same set of weights in a small kernel is reused for different regions
    (e.g., For the FCN, a 100x100 grayscale image with single hidden layer of 1,000 neurons
    → 10,000,000 parameters)
  - So, CNNs dramatically can reduce the number of parameters, thereby avoiding overfitting

- **Local connectivity**
  - Each neuron is connected to a local patch of the input
    → Mimicking receptive fields of the human visual cortex
  - So, model learns local patterns like edges, textures, shapes



https://sketchedge.net/en

✓Edges define shapes – crucial for recognizing outlines, boundaries, parts

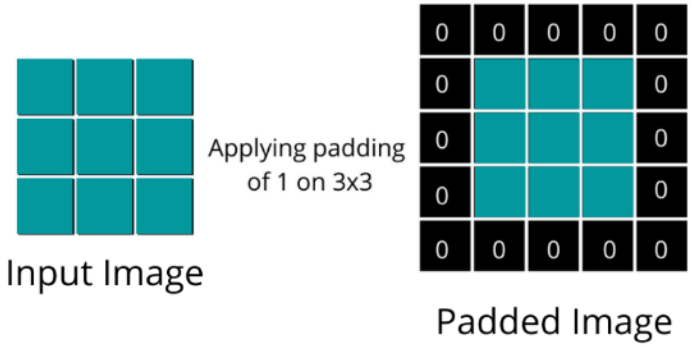
✓Color is contextual – useful for distinguishing objects
but not as essential for shape identification

- **Translation invariance**
  - Convolution detects the same pattern no matter where it appears

Padding / Stride / Pooling

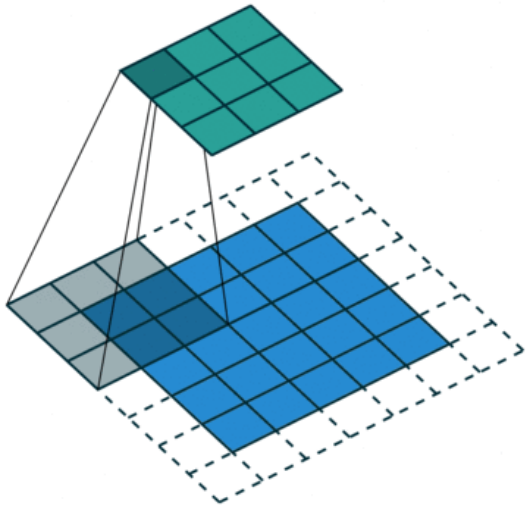
Padding

- Prevents size reduction and lets CNNs use edge information
- Allows filters to operate on border pixels



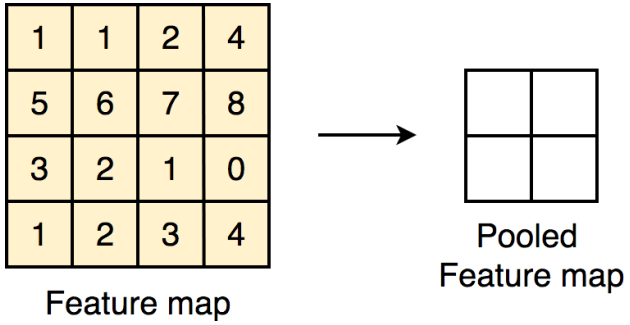
Stride

- Downscales the image by skipping positions during convolution
- Stride = 1 → full resolution feature map
- Stride = 2 → halves width and height



Pooling

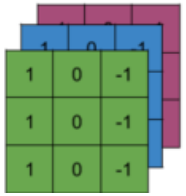
- Summarize nearby pixels to reduce size and add robustness
- Max pooling, average pooling



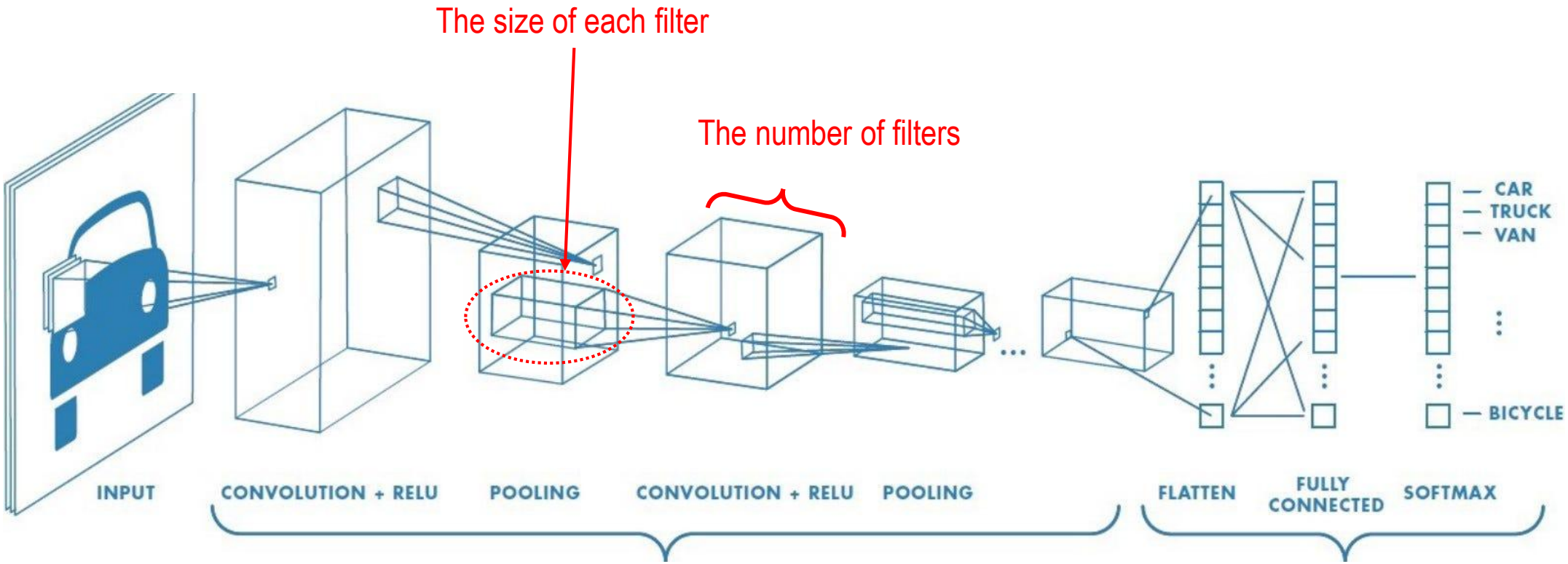
https://welldonecode.tistory.com/148

CNN architecture

https://towardsdatascience.com/from-lenet-to-efficientnet-the-evolution-of-cnns-3a57eb34672f/

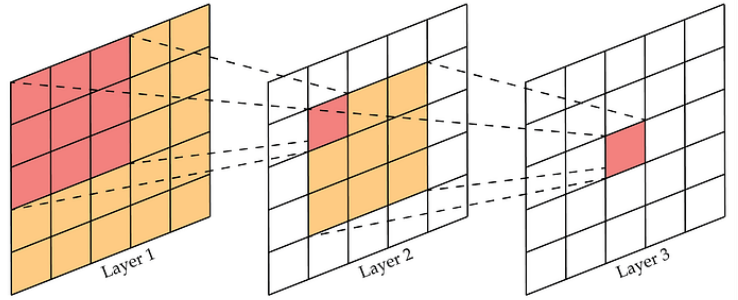


In CNNs, the **filters are what the model actually learns** during training,
But most visual explanations focus on the resulting activation maps

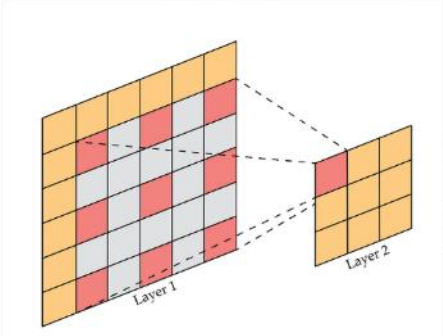


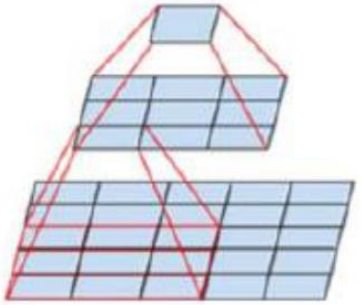
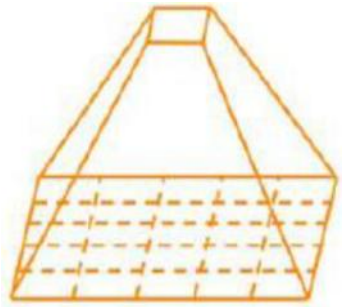
Receptive field

Two stacked 3x3 convolutions



Dilated convolution

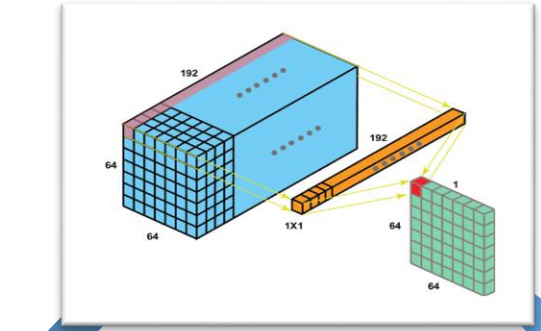


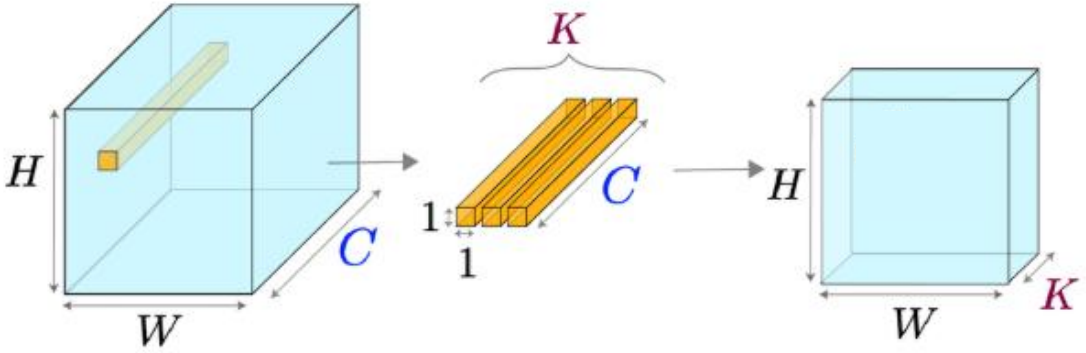
	Two statckd 3x3 Convs	One 5x5 Convs
Effective receptive field	5x5	
		
Parameters	Fewer (2x3x3)	More (5x5)
# of nonlinear activations	2	1
Computation	Faster	Slower
Feature learning	Deep	Shallow

https://medium.com/@rekalantar/receptive-fields-in-deep-convolutional-networks-43871d2ef2e9

1x1 convolution (Pointwise convolution)

- **Main roles of 1x1 convolution**
  - Mixing information across channels or acting as a feature selector
    → Reducing or increasing channels maintaining the size of feature map
  - Replacing fully connected layers





Weighted Sum

Vector and Matrix multiplication (Row-wise)

$$aC = \begin{bmatrix} a_1 & a_2 & a_3 \end{bmatrix} \begin{bmatrix} c_{11} & c_{12} & c_{13} & c_{14} \\ c_{21} & c_{22} & c_{23} & c_{24} \\ c_{31} & c_{32} & c_{33} & c_{34} \end{bmatrix}$$

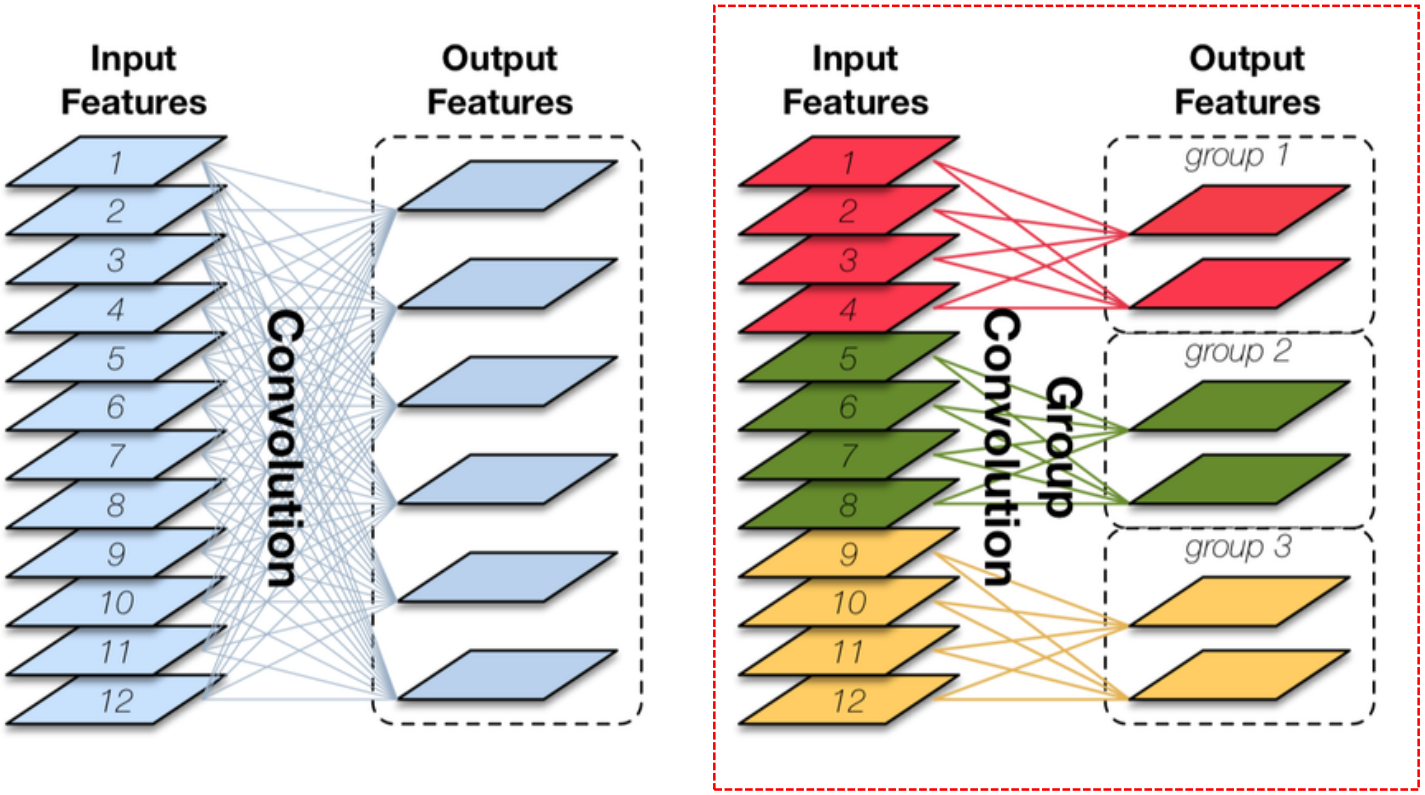
$$= a_1 r^{(1)} + a_2 r^{(2)} + a_3 r^{(3)}$$

https://cvml-expertguide.net/terms/dl/layers/convolution-layer/1x1-convolution/

Group convolution

Group convolutionhttps://www.researchgate.net/figure/CondenseNets-group-convolution-on-the-right-that-replaces-DenseNets-standard_fig5_356222219

- **Reduce computational cost**
  - Each group is convolved separately with its own set of filters → fewer total operations
- **Lower memory usage**
  - Fewer connections mean fewer weights → lighter model
- **Improve parallelization**
  - Easier to split groups across different hardware units



✓ *But, reduced cross-channel interaction*

Depthwise (separable) convolution

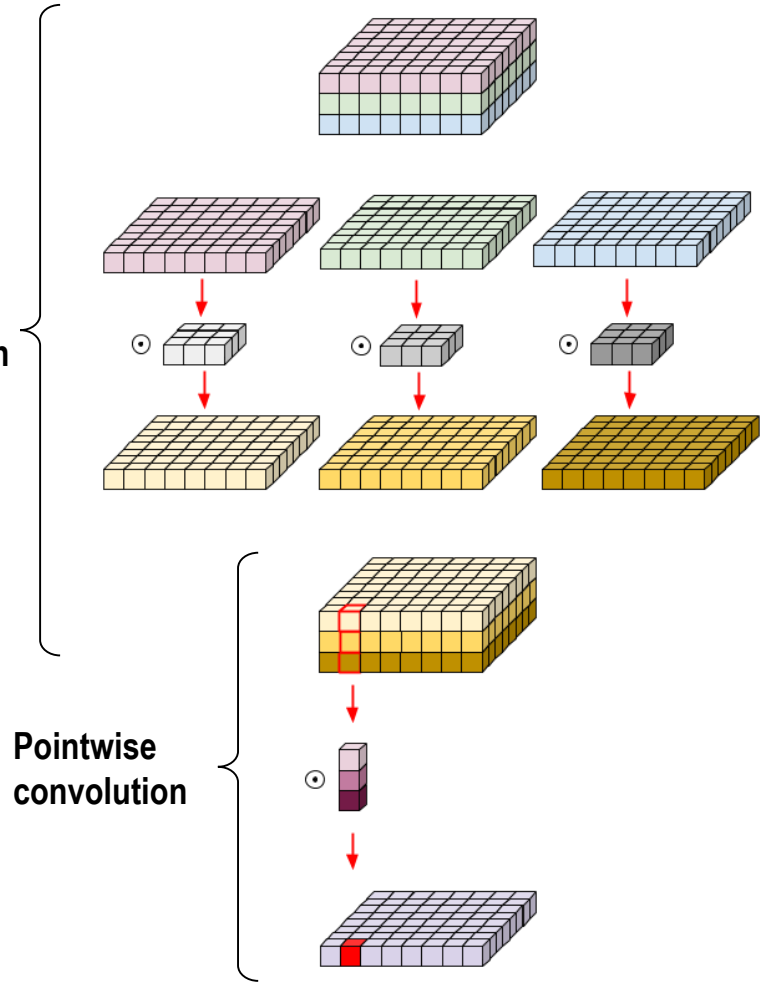
- **Depthwise convolution**
  - N filters for N input channels → reduces computation and parameters
  - Faster and more memory efficient
  - Enables real-time inference on edge devices (i.e., lightweight models like MobileNet)

✓ *No cross-channel interaction*

- **Depthwise separable convolution**
  - Typically followed by a pointwise convolution → To mix features across channels

✓ *Cross-channel interaction*

Depthwise separable convolution



Transposed convolution

https://realblack0.github.io/2020/05/11/transpose-convolution.html
https://viso.ai/deep-learning/convolution-operations/
https://gaussian37.github.io/dl-concept-checkboard_artifact/

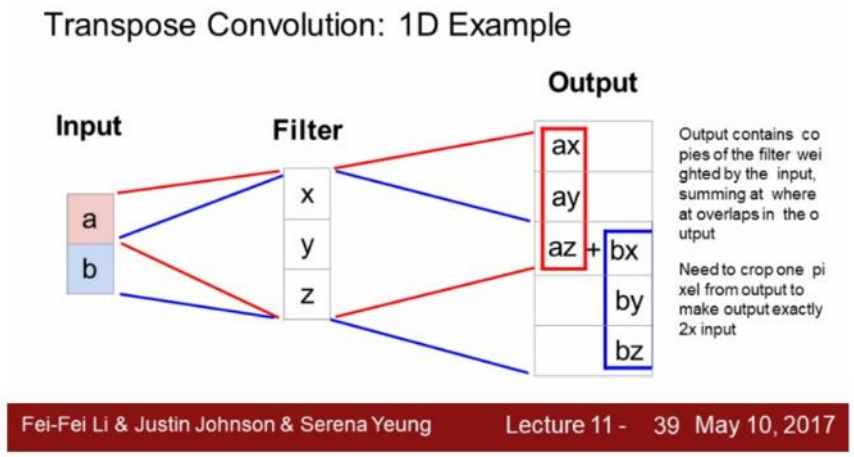
- **Goal**
  - Increasing the spatial resolution of a feature map (i.e., upsampling)
    - Used in various model architectures like Unet, FCN, GAN, etc.
- **Key contribution**
  - learning how to upsample intelligently → Learnable upsampling

But, checkerboard artifacts are detected

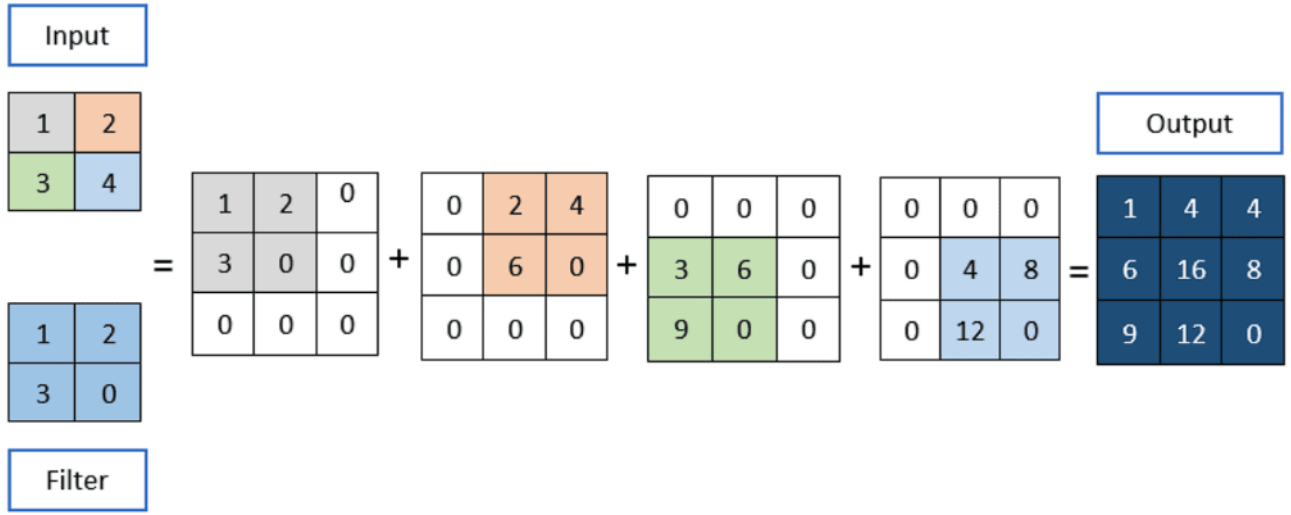


Salimans et al., 2016 [2]

1 Dimensional



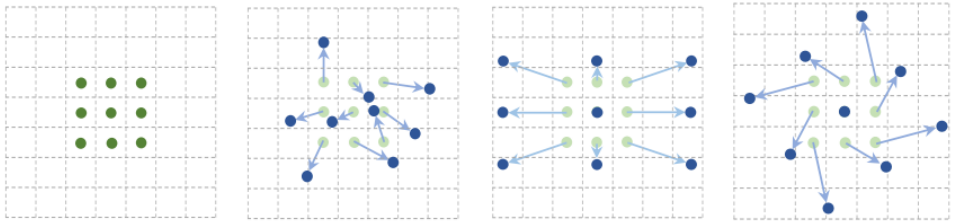
2 Dimensional



[Deformable Convolutional Networks]

- **Limitation of the standard CNN**
  - Using fixed sampling grid in convolution
    → Hard to model large or unknown geometric transformation

- **Key contributions**
  - Deformable convolution → Deformation of the receptive field
  - Deformable ROI pooling → Spatial pooling adapts to the object shape



(a) (b) (c) (d)

Figure 1: Illustration of the sampling locations in 3×3 standard and deformable convolutions. (a) regular sampling grid (green points) of standard convolution. (b) deformed sampling locations (dark blue points) with augmented offsets (light blue arrows) in deformable convolution. (c)(d) are special cases of (b), showing that the deformable convolution generalizes various transformations for scale, (anisotropic) aspect ratio and rotation.

Architecture (1/2)

- **Deformable convolution**

The 2D convolution consists of two steps: 1) sampling using a regular grid $\mathcal{R}$ over the input feature map $\mathbf{x}$; 2) summation of sampled values weighted by $\mathbf{w}$. The grid $\mathcal{R}$ defines the receptive field size and dilation. For example,

$$\mathcal{R} = \{(-1,-1),(-1,0),\ldots,(0,1),(1,1)\}$$

defines a 3×3 kernel with dilation 1.

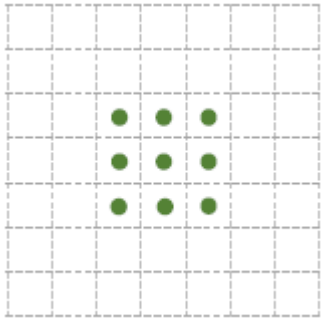
For each location $\mathbf{p}_0$ on the output feature map $\mathbf{y}$, we have

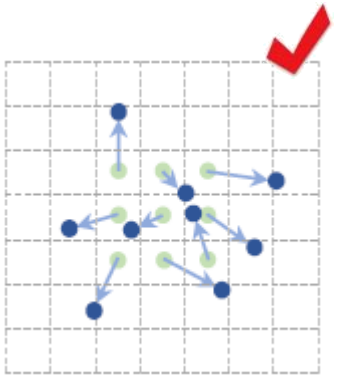
$$\mathbf{y}(\mathbf{p}_0) = \sum_{\mathbf{p}_n \in \mathcal{R}} \mathbf{w}(\mathbf{p}_n) \cdot \mathbf{x}(\mathbf{p}_0 + \mathbf{p}_n), \quad (1)$$

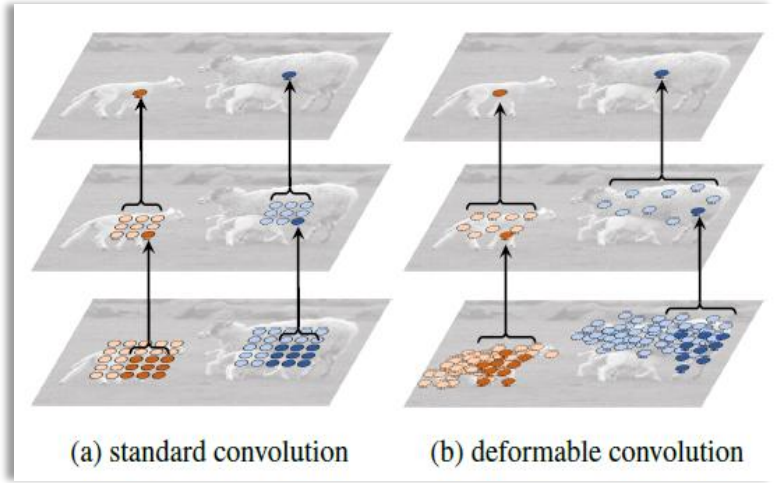
where $\mathbf{p}_n$ enumerates the locations in $\mathcal{R}$.

In deformable convolution, the regular grid $\mathcal{R}$ is augmented with offsets $\{\Delta\mathbf{p}_n | n = 1, ..., N\}$, where $N = |\mathcal{R}|$. Eq. (1) becomes

$$\mathbf{y}(\mathbf{p}_0) = \sum_{\mathbf{p}_n \in \mathcal{R}} \mathbf{w}(\mathbf{p}_n) \cdot \mathbf{x}(\mathbf{p}_0 + \mathbf{p}_n + \Delta\mathbf{p}_n). \quad (2)$$







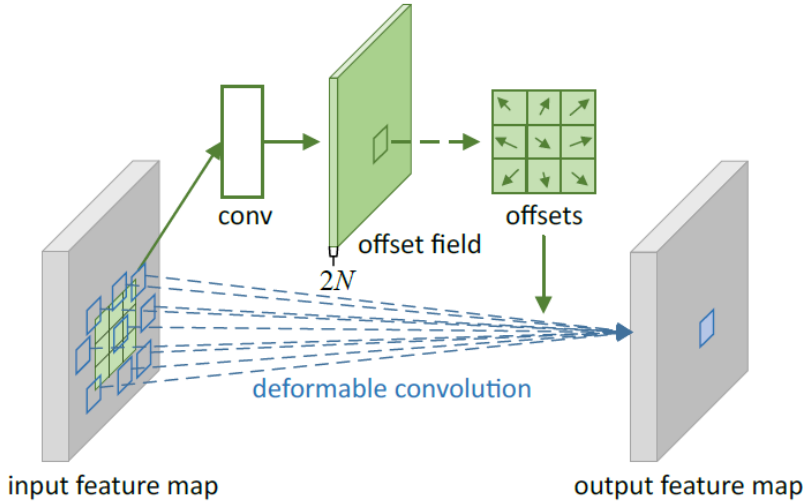


Figure 2: Illustration of 3×3 deformable convolution.

Architecture (2/2)

- **Deformable ROI pooling**

RoI Pooling [15] Given the input feature map $\mathbf{x}$ and a RoI of size $w \times h$ and top-left corner $\mathbf{p}_0$, RoI pooling divides the RoI into $k \times k$ (k is a free parameter) bins and outputs a $k \times k$ feature map $\mathbf{y}$. For (i, j)-th bin ($0 \leq i, j < k$), we have

Average pooling

$$\mathbf{y}(i,j) = \sum_{\mathbf{p} \in bin(i,j)} \mathbf{x}(\mathbf{p}_0 + \mathbf{p})/n_{ij}, \quad (5)$$

where n_{ij} is the number of pixels in the bin. The (i, j)-th bin spans $\lfloor i\frac{w}{k} \rfloor \leq p_x < \lceil (i+1)\frac{w}{k} \rceil$ and $\lfloor j\frac{h}{k} \rfloor \leq p_y < \lceil (j+1)\frac{h}{k} \rceil$.

Similarly as in Eq. (2), in deformable RoI pooling, offsets $\{\Delta\mathbf{p}_{ij} | 0 \leq i, j < k\}$ are added to the spatial binning positions. Eq.(5) becomes

$$\mathbf{y}(i,j) = \sum_{\mathbf{p} \in bin(i,j)} \mathbf{x}(\mathbf{p}_0 + \mathbf{p} + \Delta\mathbf{p}_{ij})/n_{ij}. \quad (6)$$

Typically, $\Delta\mathbf{p}_{ij}$ is fractional. Eq. (6) is implemented by bilinear interpolation via Eq. (3) and (4).

Region proposal + ROI pooling

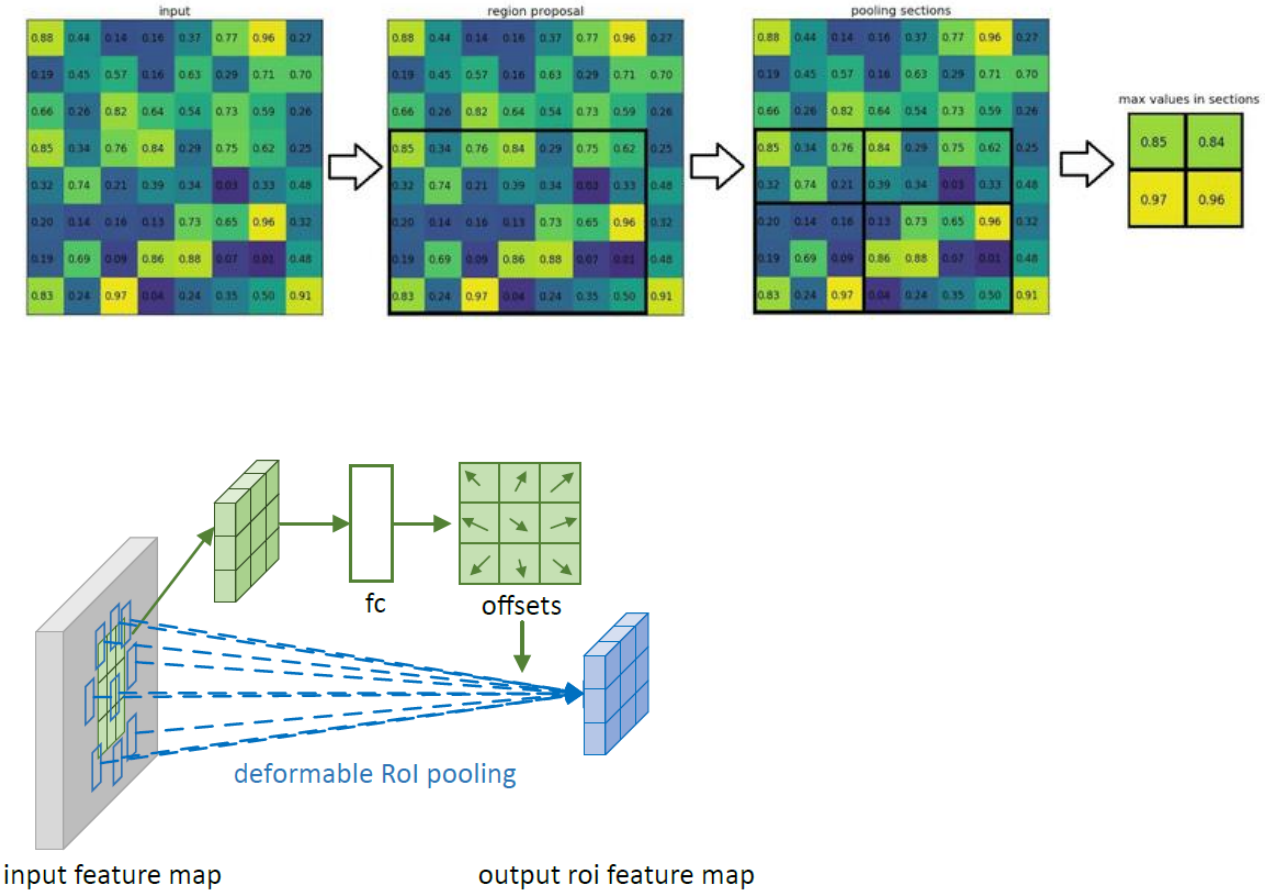


Figure 3: Illustration of 3×3 deformable RoI pooling.

Experiments

- **Deformable convolution**



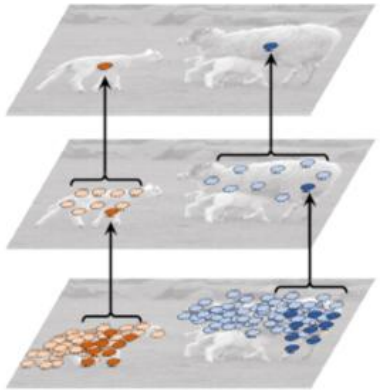


Figure 6: Each image triplet shows the sampling locations ($9^3 = 729$ red points in each image) in three levels of 3×3 deformable filters (see Figure 5 as a reference) for three activation units (green points) on the background (left), a small object (middle), and a large object (right), respectively.

- **Deformable ROI pooling**

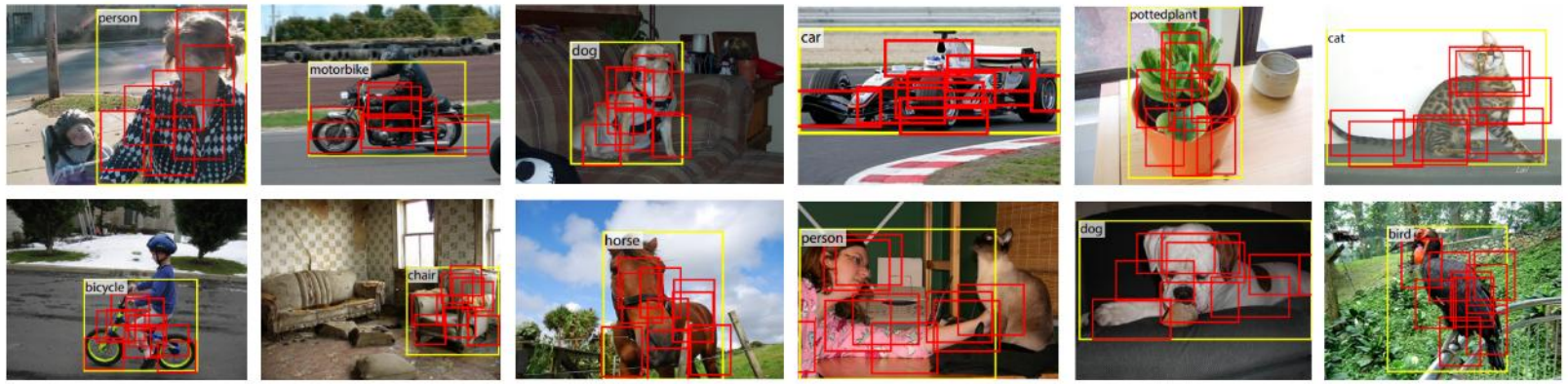
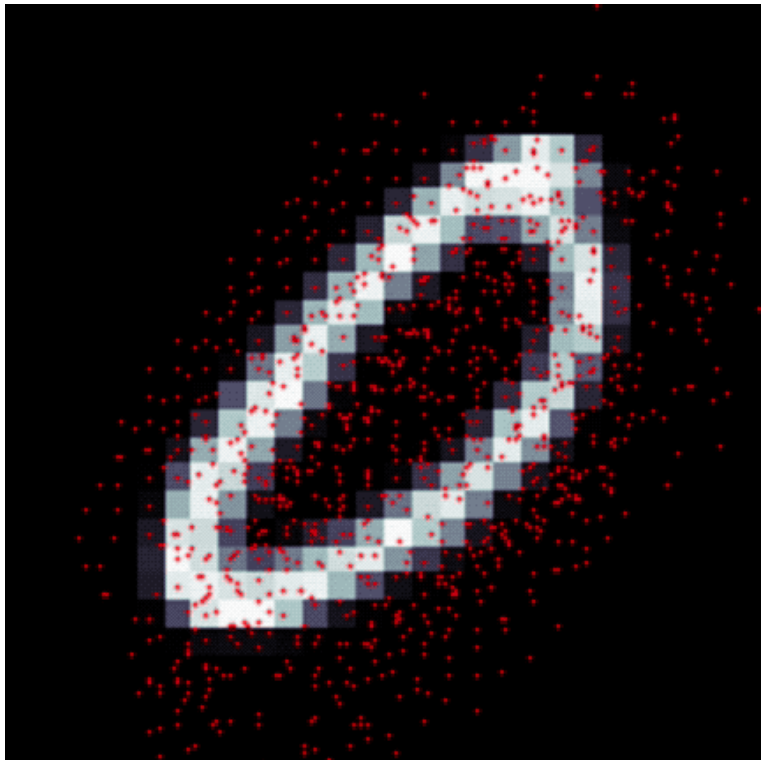


Figure 7: Illustration of offset parts in deformable (positive sensitive) RoI pooling in R-FCN [7] and 3×3 bins (red) for an input RoI (yellow). Note how the parts are offset to cover the non-rigid objects.

Demo

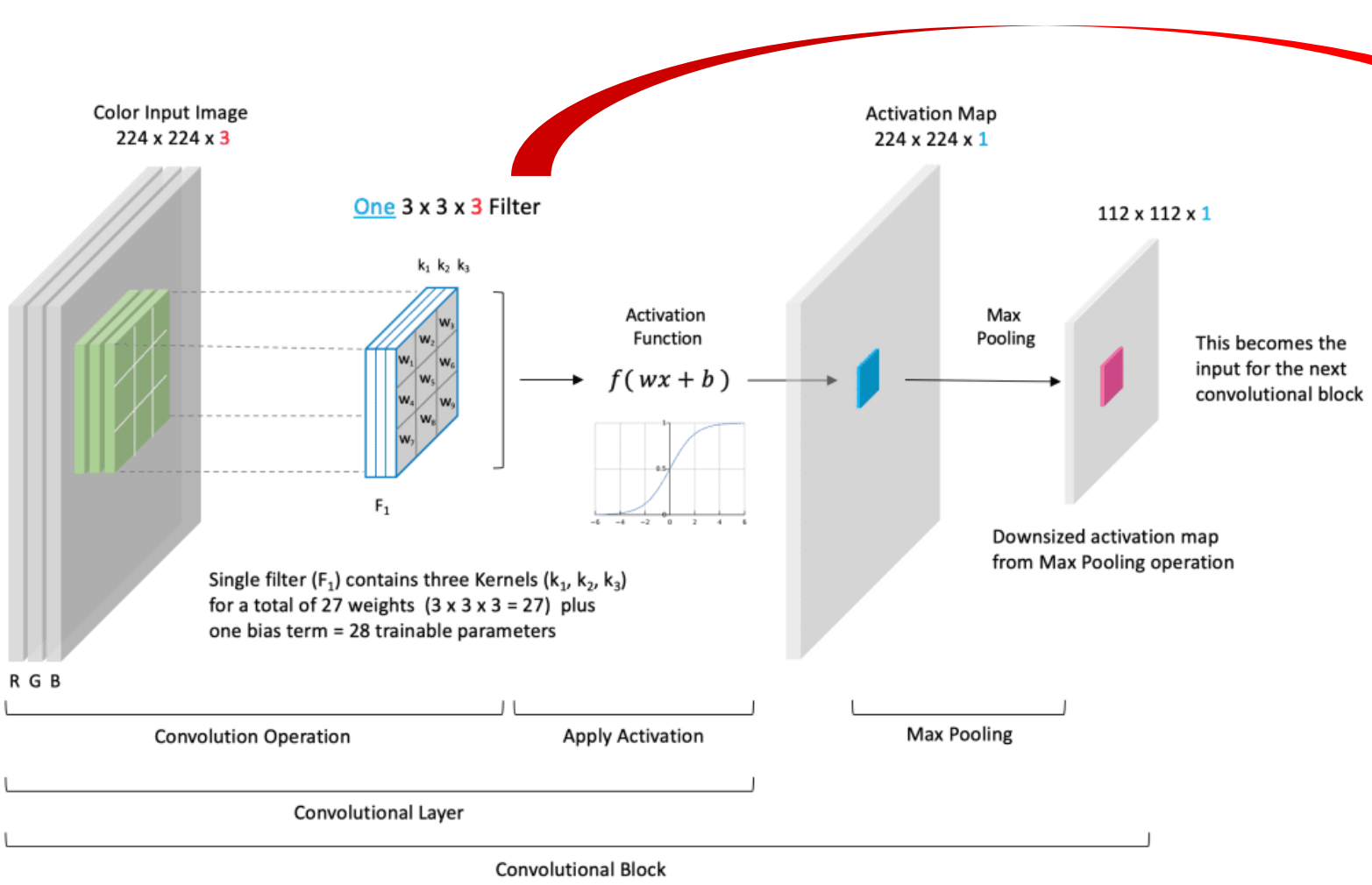


Test Accuracy	Regular CNN	Deformable CNN
Regular MNIST	98.74%	97.27%
Scaled MNIST	57.01%	92.55%

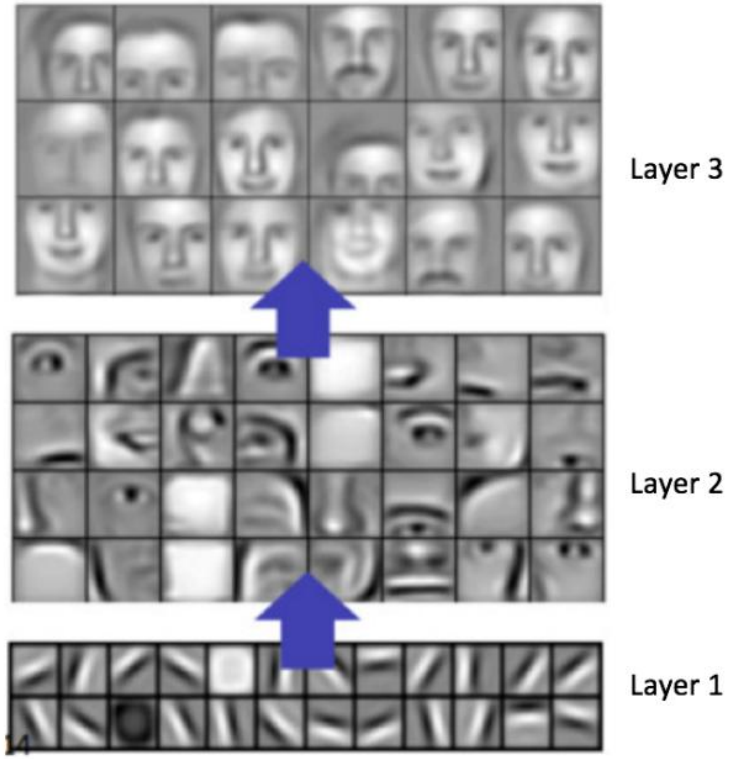
https://github.com/kastnerkyle/deform-conv
https://www.youtube.com/watch?v=yGFVO2B8gok

What are learned from CNN ?

- **A CNN learns filters** that can detect edges, textures, shapes, and object parts

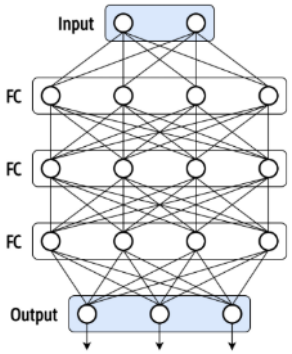
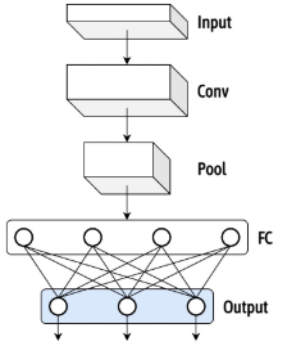


Effective filter



https://learnopencv.com/understanding-convolutional-neural-networks-cnn/
https://www.kdnuggets.com/2016/11/intuitive-explanation-convolutional-neural-networks.html/3

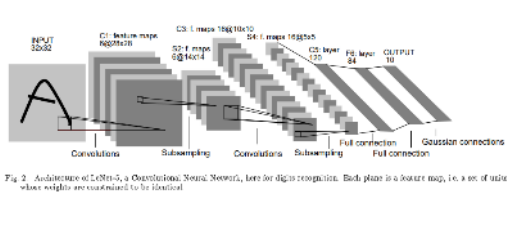
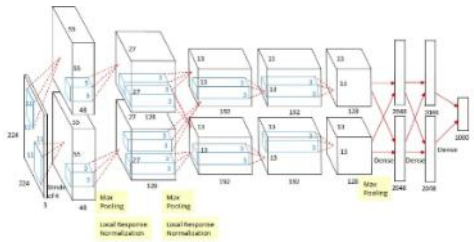
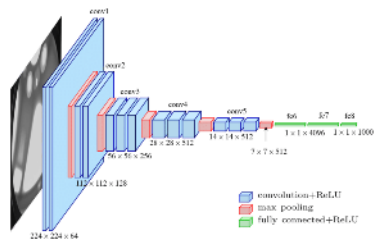
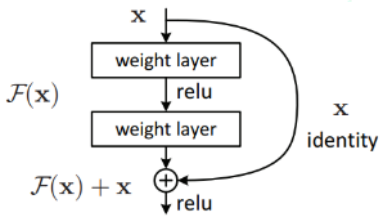
Fully Connected Networks vs. Convolutional Neural Network

	FCN (Fully Connected Networks)	CNN (Convolutional Neural Network)
Connection pattern	All nodes	Local regions (i.e., receptive field)
Weight	No weight sharing (unique weights per connections)	Weight sharing
Spatial structure	Ignores spatial locality	Preserves spatial locality
Overfitting risk	High (a lot of parameters)	Low
Interpretability	Hard to interpret learned features	Interpretable well especially in lower layers
Typical uses	Classification head MLP models	Vision tasks Feature extraction
Summary	✓ Learn global patterns ✓ Require many weights ✓ Ignore spatial information 	✓ Learn local spatial patterns ✓ Reuse weights via convolution ✓ Efficient for images 

https://wikidocs.net/164365

Evolution of CNN

CNN models

	LeNet-5	AlexNet	VGGNet	ResNet
Year	1998	2012	2014	2015
Motivation	MLP limitations (Too many parameters)	Large-scale learning (Utilize GPU acceleration)	Deeper networks (Stack of 3x3 conv.)	Degradation problem
Input Size	32×32	224×224	224×224	224×224
Layers	7	8	16/19	18–152+
Activation	Tanh	ReLU	ReLU	ReLU
Innovation	Weight sharing	GPU, Dropout	Uniform 3×3 Conv	Residual connections
Parameters	Tens of thousands	~60M	~138M	25M–60M+
Training Difficulty	Low	Medium	High	Lower (for deeper nets)
Performance	Low accuracy	ImageNet winner	High, but heavy	State-of-the-art
Contributions	First practical CNN model	Massive leap in performance	Simple and deep model	Skip connections
Model diagram				

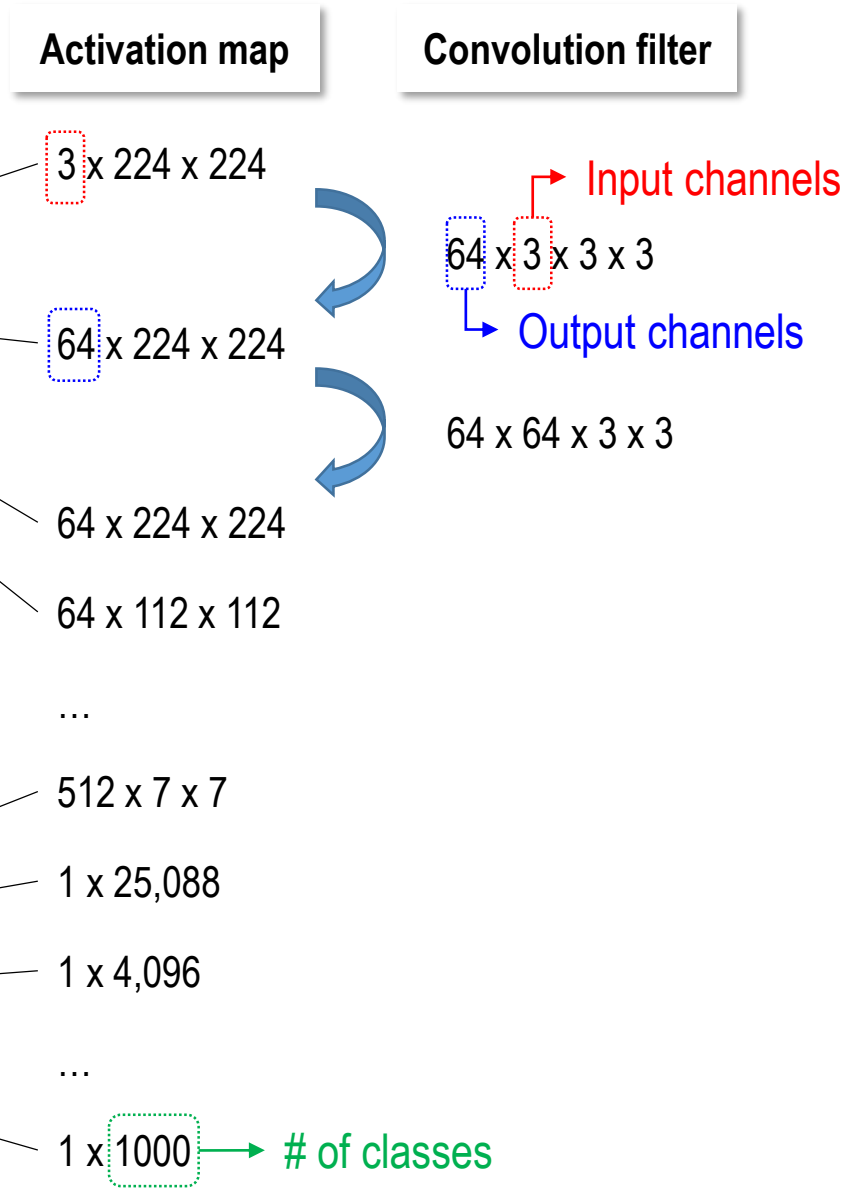
[VGGNet]

Table 1: **ConvNet configurations** (shown in columns). The depth of the configurations increases from the left (A) to the right (E), as more layers are added (the added layers are shown in bold). The convolutional layer parameters are denoted as "conv⟨receptive field size⟩-⟨number of channels⟩". The ReLU activation function is not shown for brevity.

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 **conv3-64**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 **conv3-128**	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 **conv3-256**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 **conv3-512**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 **conv3-512**
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Table 2: **Number of parameters** (in millions).

Network	A,A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144



[ResNet]

- **Why error increases as the number of layers grows (i.e., so-called degradation problem)** ?
  - Overfitting ? → Even in training, deeper plain network has higher error than the shallower one
  - Vanishing gradient ? → To some extent, largely resolved by batch normalization, weight initialization, etc.

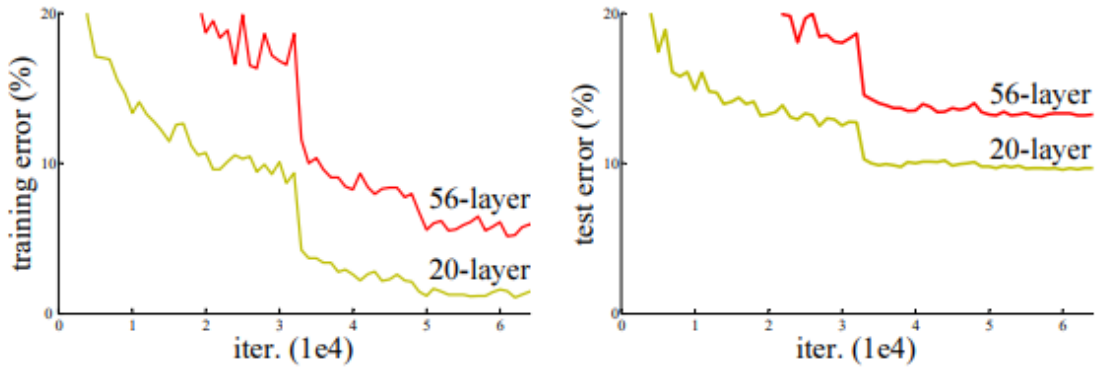
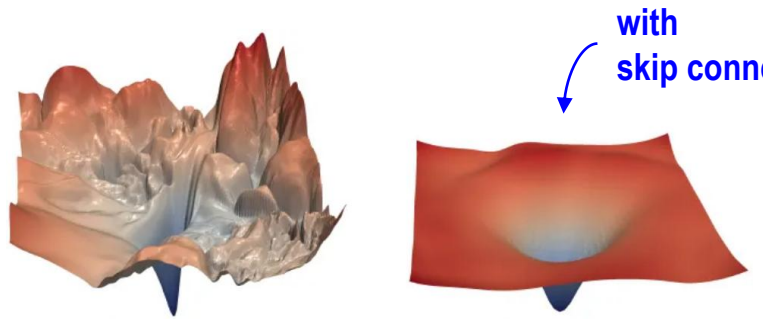


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer "plain" networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

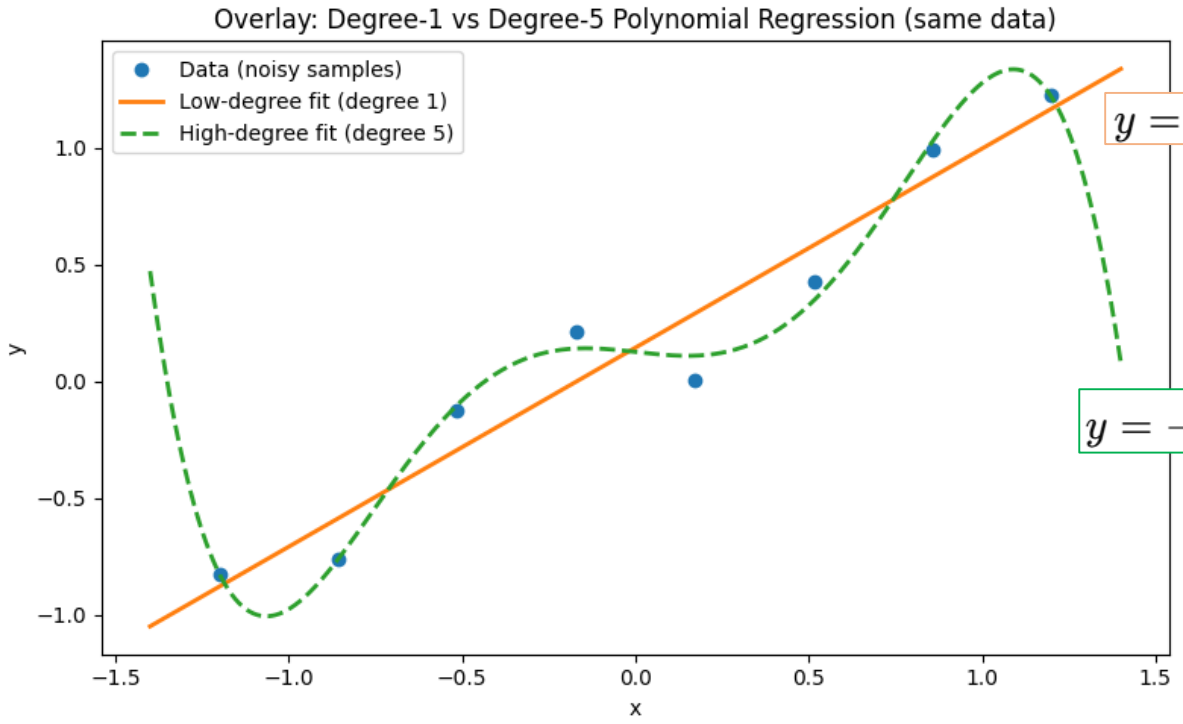
- **Optimization difficulty**
  - Learning 'Identity mapping' in a high-dimensional and non-linear space is difficult in deeper networks.

Skip connection (1/2)

- **Regularization of weights**
  - Large change of weights can increase model complexity → may lead to overfitting
  - Thus, slight change of weights are preferred → smoother optimization dynamics



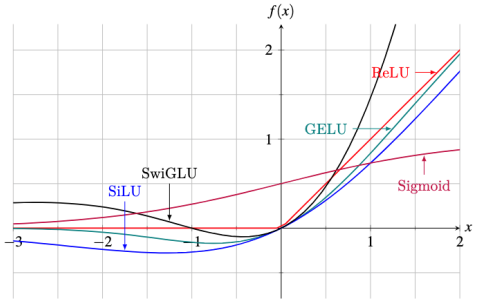
Loss landscape



$$y = 0.8528\,x + 0.1441$$

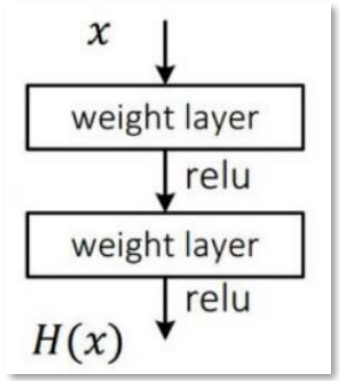
$$y = -1.333\,x^5 + 0.05351\,x^4 + 2.626\,x^3 - 0.02834\,x^2 - 0.1662\,x + 0.1256$$

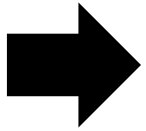
Skip connection (2/2)

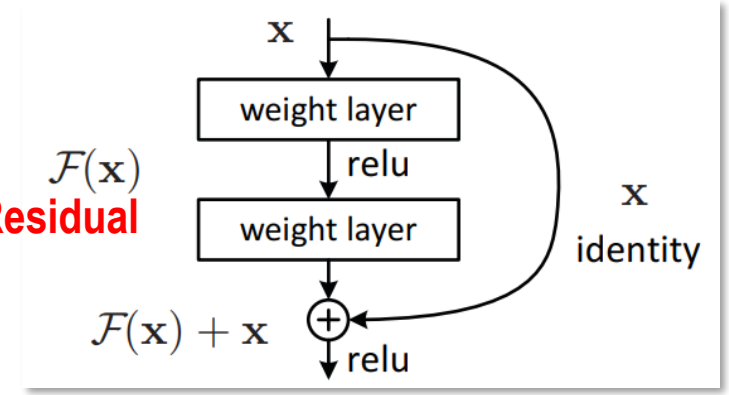


- For $H(x) \approx x$,
  - In general connection, $W_1 = W_2 = I$
  - In skip connection, $W_1 = W_2 = \mathbf{0}$
    - Normally, **initial weights are set centered on zero (e.g., Xavier, He)** → Thus, the change of weight around 0 is easier in training
    - Non-linear region of the activation functions is around 0

- As a result, the **residual learning (by skip connections) is effective to the degradation problem**





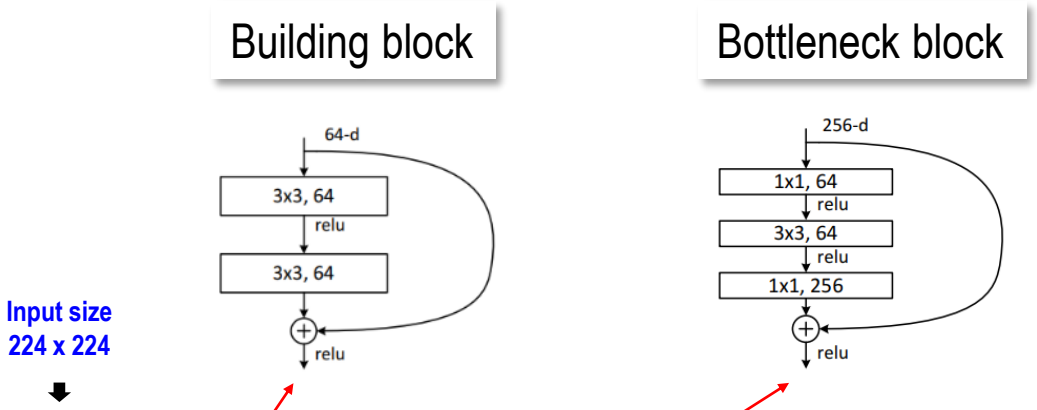


$$H(x) = \sigma(\sigma(xW_1)W_2)$$

$$H(x) := F(x) + x$$

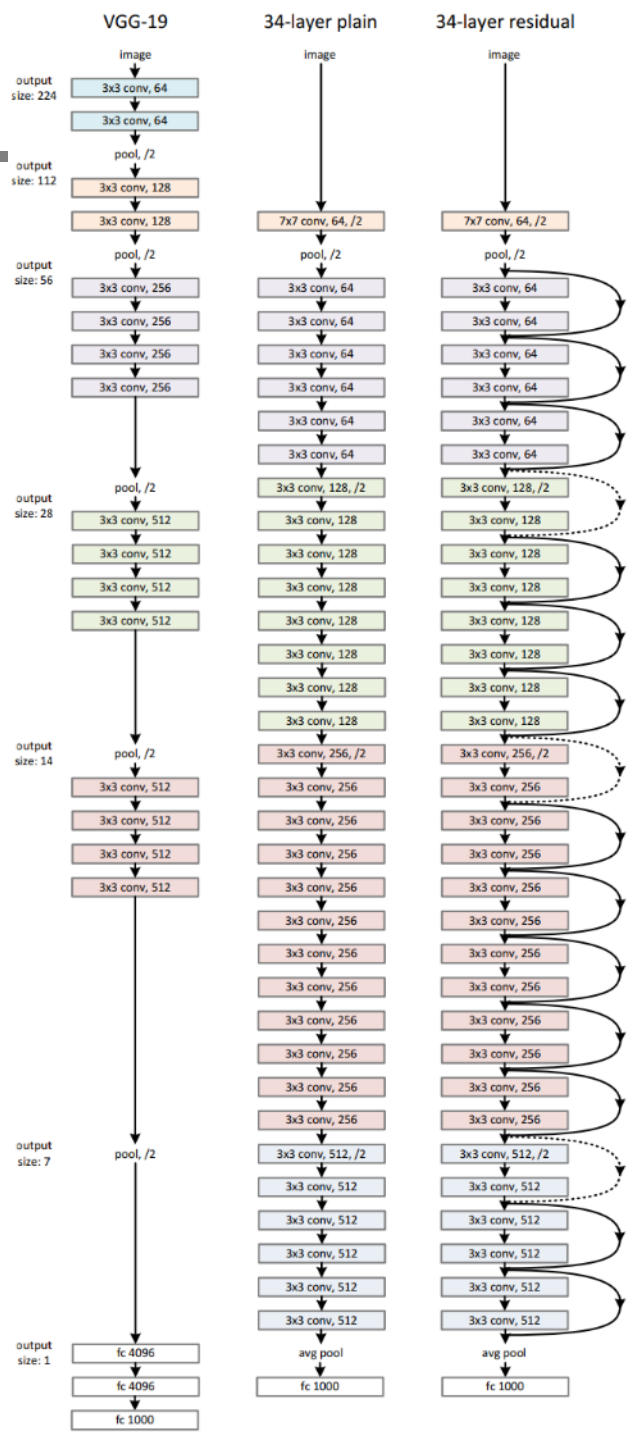
$$F(x) = \sigma(xW_1)W_2$$

Architecture



layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2 [3×3, 64; 3×3, 64] ×2	[3×3, 64; 3×3, 64] ×3	[1×1, 64; 3×3, 64; 1×1, 256] ×3	[1×1, 64; 3×3, 64; 1×1, 256] ×3	[1×1, 64; 3×3, 64; 1×1, 256] ×3
conv3_x	28×28 **Stride 2**	[3×3, 128; 3×3, 128] ×2	[3×3, 128; 3×3, 128] ×4	[1×1, 128; 3×3, 128; 1×1, 512] ×4	[1×1, 128; 3×3, 128; 1×1, 512] ×4	[1×1, 128; 3×3, 128; 1×1, 512] ×8
conv4_x	14×14 **Stride 2**	[3×3, 256; 3×3, 256] ×2	[3×3, 256; 3×3, 256] ×6	[1×1, 256; 3×3, 256; 1×1, 1024] ×6	[1×1, 256; 3×3, 256; 1×1, 1024] ×23	[1×1, 256; 3×3, 256; 1×1, 1024] ×36
conv5_x	7×7 **Stride 2**	[3×3, 512; 3×3, 512] ×2	[3×3, 512; 3×3, 512] ×3	[1×1, 512; 3×3, 512; 1×1, 2048] ×3	[1×1, 512; 3×3, 512; 1×1, 2048] ×3	[1×1, 512; 3×3, 512; 1×1, 2048] ×3
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Table 1. Architectures for ImageNet. Building blocks are shown in brackets (see also Fig. 5), with the numbers of blocks stacked. sampling is performed by conv3_1, conv4_1, and conv5_1 with a stride of 2.



Experimental results

- **Smaller variation of activation in ResNet**

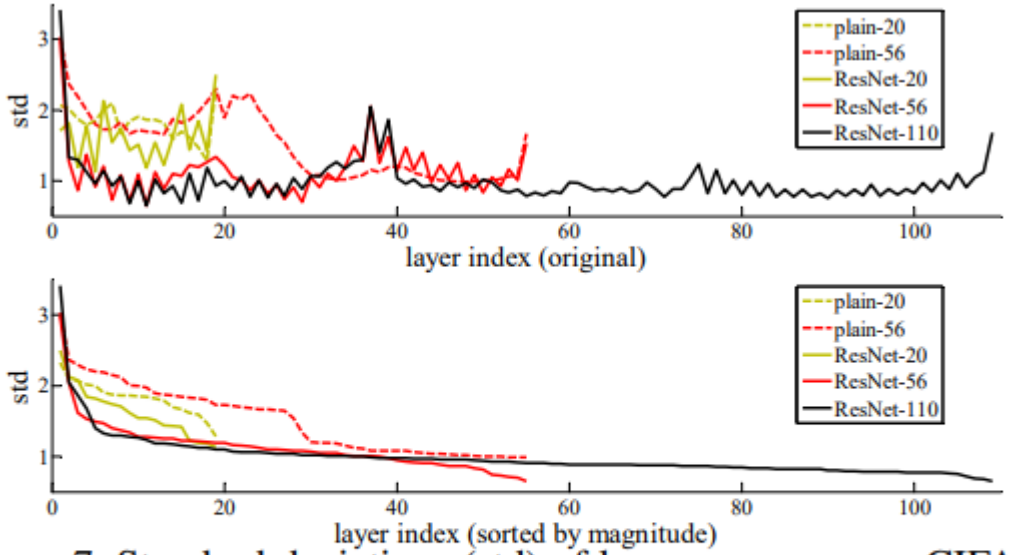


Figure 7. Standard deviations (std) of layer responses on CIFAR-10. The responses are the outputs of each 3×3 layer, after BN and before nonlinearity. **Top**: the layers are shown in their original order. **Bottom**: the responses are ranked in descending order.

method	top-5 err. (**test**)
VGG [41] (ILSVRC'14)	7.32
GoogLeNet [44] (ILSVRC'14)	6.66
VGG [41] (v5)	6.8
PReLU-net [13]	4.94
BN-inception [16]	4.82
ResNet (ILSVRC'15)	**3.57**

Table 5. Error rates (%) of **ensembles**. The top-5 error is on the test set of ImageNet and reported by the test server.

Human Visual System vs. Convolutional Neural Network

- Comparison in terms of neural networks

	Human Visual System	Convolutional Neural Network
Basic Processing Unit	Neuron with a receptive field	Convolution filter (kernel)
Early Processing Area	V1 cortex: detects edges, orientations	Conv Layer 1: learns edge detectors
Intermediate Area	V2: corners, textures	Conv Layer 2–3: corners, curves, textures
Higher Visual Area	V4: shapes, patterns	Deep Conv Layers: shapes, object parts
Object Recognition Area	IT cortex: full objects (faces, cars…)	Fully connected layers or heads
Processing Style	Hierarchical feature extraction	Sequential stacked layers (deep hierarchy)
Learning Mechanism	Plasticity, Hebbian learning	Backpropagation & gradient descent
Strengths	Generalizes from few examples	Learns patterns well Strong large-data performance
Limitations	Sometimes fooled by illusions	Adversarial vulnerabilities

